

UNIVERSIDAD DE MURCIA

FACULTAD DE INFORMÁTICA



GRADO EN INGENIERÍA INFORMÁTICA
CURSO 2018-2019

TRABAJO FIN DE GRADO

MÉTODOS DE MONTE CARLO PARA INTEGRACIÓN

José Antonio Hernández López

TUTORIZADA POR:
JOSÉ RODRÍGUEZ RUIZ

Declaración de originalidad

D. JOSÉ ANTONIO HERNÁNDEZ LÓPEZ, con DNI 48835386S, estudiante de la titulación de Grado en Ingeniería Informática de la Universidad de Murcia y autor del TF titulado "Métodos de Monte Carlo para integración".

De acuerdo con el Reglamento por el que se regulan los Trabajos Fin de Grado y de Fin de Máster en la Universidad de Murcia (aprobado C. de Gob. 30-04-2015, modificado 22-04-2016 y 28-09-2018), así como la normativa interna para la oferta, asignación, elaboración y defensa de los Trabajos Fin de Grado y Fin de Máster de las titulaciones impartidas en la Facultad de Informática de la Universidad de Murcia (aprobada en Junta de Facultad 27-11-2015).

DECLARO:

Que el Trabajo Fin de Grado presentado para su evaluación es original y de elaboración personal. Todas las fuentes utilizadas han sido debidamente citadas. Así mismo, declara que no incumple ningún contrato de confidencialidad, ni viola ningún derecho de propiedad intelectual e industrial.

En Murcia, a 24 de Junio de 2019.

Fdo.:



José Antonio Hernández López.

Resumen

El objetivo principal de este trabajo es el estudio de los métodos de Monte Carlo para el cálculo de integrales. El término "Monte Carlo" se aplica a un conjunto de métodos matemáticos que se empezaron a usar en los 1940s para el desarrollo de armas nucleares en Los Álamos, favorecidos por la aparición de los ordenadores digitales modernos. Consisten en resolver un problema mediante la invención de juegos de azar (estos métodos se llamaron de "Monte Carlo" en referencia al Casino de Montecarlo en Mónaco por ser "la capital del juego de azar") cuyo comportamiento simula algún fenómeno real gobernado por una distribución de probabilidad (e.g. un proceso físico) o sirve para realizar un cálculo (e.g. evaluar una integral) [4, 3, 10].

Como en este trabajo estamos interesados en calcular integrales de funciones, es natural preguntarse primero cómo se calculan áreas y volúmenes de objetos. Por ello, en el **Capítulo 1** (basado en las notas de clase [9]), introducimos el método de Monte Carlo para el cálculo de áreas en dos dimensiones. Posteriormente, extendemos lo anterior a d dimensiones e introducimos cómo se calcula el volumen de un objeto D encerrado en un rectángulo d -dimensional $R \subset \mathbb{R}^d$ que no es más que estimar la esperanza de una variable aleatoria discreta que sigue una distribución de Bernoulli (normalmente se estima considerando varias muestras y tomando la media muestral). Dicha estimación va a tener un error y, cómo es lógico, nosotros debemos dar una cota del mismo. Uno de los primeros teoremas sobre acotación de este trabajo es el Teorema 1.8 (consecuencia directa del Teorema Central del Límite) que establece que dada una confianza $\gamma \in (0, 1)$, existe un z_γ tal que si tomamos un n (número de muestras) suficientemente grande,

$$P \left(|V - V_D| \leq z_\gamma V_R^{\frac{1}{2}} \frac{\sqrt{V_D \left(1 - \frac{V_D}{V_R}\right)}}{\sqrt{n}} \right) \approx \gamma.$$

V_R es el volumen del rectángulo d -dimensional (valor conocido) y V_D es el volumen del sólido D . Es decir, para una V_{exp} estimación experimental del volumen del objeto se tiene que

$$|V_{exp} - V_D| \leq z_\gamma V_R^{\frac{1}{2}} \frac{\sqrt{V_D \left(1 - \frac{V_D}{V_R}\right)}}{\sqrt{n}}$$

con una confianza de aproximadamente γ .

Se observa que el factor $1/\sqrt{n}$ no depende de la dimensión. Por otro lado, estas dos cotas del error no son aplicables en la práctica, pues V_D no es conocido (¡es precisamente lo que queremos calcular!). Así pues, este primer capítulo concluye con una alternativa a la cota anterior (Teorema 1.13).

Una vez estudiado el cálculo de volumen d -dimensional pasamos a estar interesados en calcular la integral de una función:

$$I_f = \int_R f(x) dx$$

donde $f : R \subset \mathbb{R}^d \rightarrow \mathbb{R}$ y R es un rectángulo d -dimensional. Normalmente se supondrá $f \in L^2(R)$. En el **Capítulo 2** (basado en las notas de clase [9], el libro [3] y en el artículo [8]) definimos un concepto clave: **Método de Monte Carlo para integración** o **MI** que no es más que una variable aleatoria M tal que su esperanza es I_f . Dicha variable aleatoria tiene lo que se denomina un **estimador de la integral** o **EMI** que no es más que la media muestral $\bar{M}$ de un conjunto de n muestras. Usando, como en el cálculo de volúmenes, el Teorema Central del Límite obtenemos una cota teórica del error (Teorema 2.3):

$$\lim_n P \left(|\bar{M} - I_f| \leq z_\gamma \sqrt{\text{Var}[M] \cdot n^{-1/2}} \right) = \gamma.$$

La varianza de M es desconocida, por ello en la Proposición 2.6 se da una alternativa utilizando la varianza muestral (un estimador de $\text{Var}[M]$). Dados dos EMIs, ¿cómo los comparamos? La forma de comparar dos métodos viene dada en la definición de exactitud presente en la Definición 2.7: un método es más exacto que otro si y solo si su varianza es menor.

Una vez definidos de manera formal los MIs, se introducen dos métodos utilizados:

- El método de integración de *Acierto o fallo* que reduce el cálculo de la integral al cálculo del volumen bajo la curva.
- El método de *Media en el muestreo* que consiste en $S = V_R \cdot Y$, donde V_R es el volumen del dominio de f e $Y = f(X)$ con $X \sim U(\text{int}(R))$ (variable aleatoria uniforme en el interior de R).

El método de *Media en el muestreo* es más exacto que *Acierto o fallo* según la definición de exactitud antes comentada.

Tanto el **Capítulo 1** como el **Capítulo 2** describen unos métodos muy ilustrativos pero que no se suelen utilizar debido a que tanto el error real como la estimación teórica del error son bastante malos salvo que se aumente mucho el número de muestras. Este hecho se ve claramente en los ejemplos expuestos en susodichos capítulos. Así pues, en el **Capítulo 3** (basado en el libro [3] y en el artículo [12]) se refinan los métodos de Monte Carlo básicos ya vistos con los llamados métodos de reducción de varianza. Antes de empezar con dichos métodos, es necesario generalizar *Media en el muestreo* obteniendo así la técnica *Media en el muestreo generalizado*. Este MI viene dado por la variable aleatoria $Y = \frac{f(X)}{\omega(X)}$ donde X es una variable aleatoria continua con función de densidad ω cuyo soporte contiene al soporte de f (a ω se le conoce como *importancia en el muestreo*). El método de reducción de varianza basado en optimizar la importancia en el muestreo sostiene que la elección de ω influye mucho en la exactitud de *Media en el muestreo generalizado* (esto efectivamente es así y se ha comprobado experimentalmente en el Ejemplo 3.5); así pues, nos planteamos buscar el ω óptimo, esto es el que minimiza la varianza de Y . En el Teorema 3.6 se demuestra que la función de densidad óptima es:

$$q(x) = \frac{|f(x)|}{\int_R |f(z)| dz}.$$

Una consecuencia directa de este teorema es que la masa de probabilidad de la función debe ser mayor en las zonas donde $|f(x)|$ sea grande.

El segundo método de reducción de varianza estudiado es el muestreo estratificado. Grosso modo, este método consiste en particionar el rectángulo d -dimensional donde está definida f en lo que se denominan estratos; en cada uno de los estratos consideramos una función de densidad ω_i , un entero n_i y el EMI $\bar{Y}_{n_i}$ del método *Media en el muestreo generalizado*. Así pues, el estimador de muestreo estratificado se define como:

$$\bar{Y}_{st} = \sum_i \bar{Y}_{n_i}.$$

En la Proposición 3.12 se establece una cota del error. Y, en el Teorema 3.13, se establece un criterio para seleccionar los n_i .

El último método de reducción de varianza que vamos a ver es *variables de control*. La idea es sencilla: supongamos que tenemos una función g que aproxima f razonablemente bien de la cual conocemos su integral sobre R . Entonces mediante unos sencillos cálculos podemos ver que:

$$\int_R f(x) dx = \alpha \int_R g(x) dx + \int_R f(x) - \alpha g(x) dx = \alpha \int_R g(x) dx + E \left[\frac{f(X) - \alpha g(X)}{\omega(X)} \right].$$

El Teorema 3.17 indica el α que minimiza $Var \left[\frac{f(X) - \alpha g(X)}{\omega(X)} \right]$. Y que, dicho α , se puede estimar mediante unos estadísticos adecuados.

Una vez vistos los métodos de reducción de varianza, en el **Capítulo 4** se introducen cuatro algoritmos de integración. Cada uno de ellos está basado en un método de reducción de los mencionados anteriormente.

El primero de ellos es **VEGAS** [5, 6, 7, 10]. Este algoritmo iterativo está basado en muestreo adaptativo (en cada iteración va *aprendiendo* de la función que integra) y en la importancia en el muestreo óptima. En una dimensión, VEGAS hace una partición del intervalo ($x_0 < x_1 < \dots < x_N$, con N un número natural) y busca refinar una función de densidad que tiene la siguiente forma:

$$q(x) = \sum_{i=0}^{N-1} \frac{1}{N(x_{i+1} - x_i)} \chi_{[x_i, x_{i+1})}(x). \quad (1)$$

Así pues, para llevar a cabo dicho refinamiento solo tiene que modificar las longitudes de los intervalos. En cada iteración, se realiza una estimación de la integral y del error. Finalmente, cuando se ha llegado a un número máximo de iteraciones, utilizando todas las estimaciones generadas anteriormente, se da una estimación final de la integral. Para integrales d -dimensionales, se emplea una función de densidad de variables separadas: $q(x_1, \dots, x_d) = \prod_{i=1}^d q_i(x_i)$, donde $q_i(x_i)$ tiene la forma de la función (2).

El segundo algoritmo estudiado es **MISER** [11, 15, 10]. Este algoritmo recursivo está basado en el muestreo estratificado. El procedimiento recibe como parámetro la función, la región donde se quiere calcular la integral y el número de muestras disponibles, entonces:

1. Si el número de puntos es menor que un umbral entonces devuelve la estimación de la integral en la región de entrada usando el número de puntos introducidos como parámetro.
2. En otro caso, divide la región en dos partes, divide los puntos disponibles en dos y llama, de manera recursiva, al procedimiento en cada una de las dos regiones.

El tercer algoritmo estudiado hace uso de **árboles de regresión** [2]. Dado un conjunto de muestras $\{(x_t, f(x_t))\}_t$ se busca obtener una función simple g que aproxime a f :

$$g(x) = \sum_{i=1}^k a_i \chi_{R_i}(x),$$

donde R_i son rectángulos que particionan el dominio de f . Se toma entonces la siguiente función de densidad:

$$q_g(x) = \frac{|g(x)|}{\int_R |g(z)| dz} = \frac{1}{\sum_{i=1}^k |a_i| V_{R_i}} \sum_{i=1}^k |a_i| \chi_{R_i}(x).$$

Seguindo la densidad q_g se genera un nuevo conjunto de muestras que se utilizarán para estimar la integral (la estimación se hace utilizando la *Media en el muestreo generalizado* donde $X \sim q_g$). La distribución inducida por q_g es sencilla y, generar una muestra siguiendo dicha distribución equivale a seleccionar un rectángulo con probabilidad proporcional a $|a_i|V_{R_i}$ y luego tomar una muestra siguiendo una distribución $U(int(R_i))$. El objetivo principal de este método es mejorar la estimación de la integral sacrificando un conjunto de muestras para obtener la función g (las cuales, no se emplearán para realizar la estimación). Grosso modo, se realiza lo siguiente:

1. Se genera una serie de muestras iniciales siguiendo una distribución uniforme.
2. Se entrena el árbol varias veces con el objetivo de refinar g .
3. Utilizando $X \sim q_g$, se aplica *Media en el muestreo generalizado*.

El cuarto y último algoritmo estudiado es conocido por ***StackMonteCarlo*** [12]. Realmente es un método de post-procesamiento (pues parte de un conjunto de muestras generadas siguiendo una determinada distribución $X \sim \omega$), explota el método de reducción de varianza *variables de control* y utiliza la validación cruzada de k pliegues y el aprendizaje supervisado. Así pues dado un conjunto de datos generados siguiendo ω , este método describe una forma de obtener g y estimar α a partir de dicho conjunto. En el Ejemplo 4.6 se compara *StackMonteCarlo* con *Media en el muestreo*.

Finalmente, en la Sección 4.5 se comparan los métodos: VEGAS, MISER, árbol de regresión y *Media en el muestreo*. Para ello se utilizan cuatro funciones definidas en el rectángulo $[0, 1]^d$: la primera es sencilla de integrar, la segunda no es cuadrado integrable, la tercera es producto de exponenciales y la cuarta es una suma de funciones trigonométricas. Dibujamos las gráficas del error absoluto real y del error absoluto estimado y establecemos conclusiones.

Por último, cabe destacar que en el Anexo 6 se encuentran herramientas de probabilidad y estadística que utilizaremos a lo largo del trabajo. También se incluye el Anexo 7 con código en *python* que implementa:

- Método de Monte Carlo de cálculo de volúmenes, desarrollado en el Capítulo 1.
- *Media en el muestreo*, desarrollado en el Capítulo 2.
- *Acierto o fallo*, desarrollado en el Capítulo 2.
- Muestreo estratificado, desarrollado en el Capítulo 3.
- Integración con árboles de regresión, desarrollado en el Capítulo 4.
- *StackMonteCarlo* con árboles de regresión, desarrollado en el Capítulo 4.

Los métodos no implementados son VEGAS (puede descargarse en [6]) y MISER (puede descargarse en [15]).

Todos los métodos implementados en el Anexo 7 devuelven, al menos, la integral o volumen y una estimación del error con una confianza del 99 %.

Las librerías de *python* más importantes empleadas son: *math* [21], *numpy* [19], *sklearn* [16] (para los árboles de regresión y la validación cruzada de k pliegues) y *matplotlib* [20] para las gráficas.

Extended abstract

The main aim of this work is the study of the Monte Carlo methods for the numerical computation of integrals. The term "Monte Carlo" is applied to a set of mathematical methods which began to be used in the 1940s for the development of nuclear weapons in Los Álamos, favoured by the appearance of modern digital computers. They are implemented to solve a problem through games of chance (these methods were called "Monte Carlo" due to the Monte Carlo Casino in Monaco, the "capital of gambling"), whose behaviour simulates some real phenomena governed by a probability distribution (e.g. a physical process), or it is used to calculate something (e.g. evaluate an integral) [4, 3, 10].

Since we are interested in calculating integrals of functions, it is necessary first to find out how to calculate the area and volume of objects. Therefore, in **Chapter 1** (based on the class notes [9]), we will introduce the Monte Carlo method for the calculation of areas in two dimensions. Later on, we will extend that method to d dimensions and we will introduce the calculation of the volume of an object D inside a d -dimensional rectangle $R \subset \mathbb{R}^d$, which is merely the estimation of the expected value of a random discrete variable which follows a Bernoulli distribution (normally it is estimated by taking into account several samples and by taking the sample mean). That estimation will have an error and, naturally, we must provide an upper bound of that error. One of the first theorems about error estimation in this work is Theorem 1.8 (direct consequence of the Central Limit Theorem), which establishes that given a confidence level $\gamma \in (0, 1)$, there exists a z_γ such that, if we take a large enough n (number of samples),

$$P \left(|V - V_D| \leq z_\gamma V_R^{\frac{1}{2}} \frac{\sqrt{V_D \left(1 - \frac{V_D}{V_R}\right)}}{\sqrt{n}} \right) \approx \gamma.$$

V_R is the volume of the d -dimensional rectangle (known value) and V_D is the volume of the solid D , i.e. for an experimental estimation V_{exp} of the volume of the object, we get

$$|V_{exp} - V_D| \leq z_\gamma V_R^{\frac{1}{2}} \frac{\sqrt{V_D \left(1 - \frac{V_D}{V_R}\right)}}{\sqrt{n}}$$

with a confidence level of approximately γ .

We observe that the factor $1/\sqrt{n}$ does not depend on the dimension. On the other hand, these two error bounds are not applicable in practice, since V_D is not known (it is precisely what we want to calculate). Therefore, this first chapter concludes with an alternative to the previous bound (Theorem 1.13).

Once we have analysed the calculation of the d -dimensional volume, we focus on calculating the integral of a function:

$$I_f = \int_R f(x) dx$$

where $f : R \subset \mathbb{R}^d \rightarrow \mathbb{R}$ and R is a d -dimensional rectangle. Normally, we will assume $f \in L^2(R)$ (this hypothesis is necessary since it allows us to apply Central Limit Theorem when establishing an error bound). In **Chapter 2** (based on the class notes [9], the book [3] and the article [8]), we define a key concept: **Monte Carlo Integration** or **MI**, which is essentially a random variable M such that its expected value is I_f . This random variable has what is known as **Integral Estimator** or **EMI**, which is the sample mean $\bar{M}$ of a set of n samples. By using the Central Limit Theorem, as in the calculation of volumes, we obtain a theoretical error bound (Theorem 2.3):

$$\lim_n P \left(|\bar{M} - I_f| \leq z_\gamma \sqrt{\text{Var}[M] \cdot n^{-1/2}} \right) = \gamma.$$

The variance of M is unknown, thus in Proposition 2.6, an alternative is given by using the sample mean (an estimator of $\text{Var}[M]$). Given two EMIs, how do we compare them? The way to compare two methods is given by the definition of accuracy given in Definition 2.7: a method is more accurate than other if and only if its variance is smaller.

Once the MIs have been formally defined, we introduce the two methods used:

- The method of integration of *Hit and miss* which reduces the calculation of the integral to the calculation of the volume under the curve.
- The method of *Sample mean* which consists in $S = V_R \cdot Y$, where V_R is the volume of the domain of f and $Y = f(X)$ with $X \sim U(\text{int}(R))$ (uniform random variable inside R).

There are two reasons why *Sample mean* is better than *Hit or miss*:

1. *Sample mean* is more accurate than *Hit or miss* according to the previously given definition. In Theorem 2.21, this statement is proved, and, in Examples 2.22 and 2.22 we see that it is empirically verified.
2. The method of *Hit or miss* requires the absolute value of the function to be bounded by a known constant. Nonetheless, *Sample mean* only requires it to be square-integrable on the integration domain.

Both Chapter 1 and Chapter 2 describe very illustrative methods, but they are not usually implemented due to the fact that both the real error and the theoretical estimation of the error are quite bad, unless the number of samples is increased significantly. This fact can be clearly seen in the examples provided in those chapters. Therefore, in **Chapter 3** (based on the book [3] and the article [12]) the basic Monte Carlo methods already mentioned are refined with the so-called variance reduction methods. Before we begin with those methods, it is necessary to generalize *Sample mean*, thus obtaining the technique *Generalized sample mean*. This MI is given by the random variable $Y = \frac{f(X)}{\omega(X)}$, where X is a continuous random variable with density function ω whose support contains the support of f (ω is known as *importance sampler*). The method of variance reduction based in optimizing the *importance sampler* maintains that the selection of ω has a major influence in the accuracy of Generalized sample mean (this is definitely so and has been experimentally verified in Example 3.5); therefore, we look for the optimum ω , which minimizes the variance of Y . In Theorem 3.6, it is proven that the optimal density function is:

$$q(x) = \frac{|f(x)|}{\int_R |f(z)| dz}.$$

One direct consequence of this theorem is that the function's probability mass must be larger in the areas where $|f(x)|$ is big.

The second variance reduction method studied is stratified sampling. Broadly speaking, this method consists in partitioning the d -dimensional rectangle where f is defined in what is known as *strata*; in each one of the *strata*, we take into account one density function ω_i , an integer n_i , and the EMI $\bar{Y}_{n_i}$ from the method *Generalized sample mean*. Therefore, the estimator of stratified sampling is defined as:

$$\bar{Y}_{st} = \sum_i \bar{Y}_{n_i}.$$

In Proposition 3.12, an error bound is established and, in Theorem 3.13, a criterion to select the n_i is given.

The last variance reduction method we are going to look through is control variates. The idea is simple: suppose we have a function g that approximates f reasonably well and whose integral on R is known. Then, through easy calculations, we can see that:

$$\int_R f(x) dx = \alpha \int_R g(x) dx + \int_R f(x) - \alpha g(x) dx = \alpha \int_R g(x) dx + E \left[\frac{f(X) - \alpha g(X)}{\omega(X)} \right].$$

Theorem 3.17 indicates the α which minimizes $Var \left[\frac{f(X) - \alpha g(X)}{\omega(X)} \right]$, and also that such α can be estimated through some adequate statistical procedures.

Once the variance reduction methods have been studied, in **Chapter 4**, four integration algorithms are introduced. Each one of them is based on one of the reduction methods previously mentioned.

The first one is **VEGAS** [5, 6, 7, 10]. This iterative algorithm is based on adaptive sampling (in each iteration it learns from the function it integrates) and on the optimal *importance sampler*. In one dimension, VEGAS makes a partition of the interval $(x_0 < x_1 < \dots < x_N)$, with N a natural number) and aims to refine a density function with the following form:

$$q(x) = \sum_{i=0}^{N-1} \frac{1}{N(x_{i+1} - x_i)} \chi_{[x_i, x_{i+1})}(x). \quad (2)$$

Therefore, in order to carry out this refinement, it only has to modify the lengths of the intervals. In each iteration, an estimation of the integral and the error is implemented. Finally, when a maximum number of iterations has been reached, by using all the estimations generated previously, a final estimation of the integral is given. For d -dimensional integrals, a density function with separate variables is used: $q(x_1, \dots, x_d) = \prod_{i=1}^d q_i(x_i)$, where $q_i(x_i)$ takes the form of the function (2).

The second algorithm which has been studied is **MISER** [11, 15, 10]. This recursive algorithm is based on stratified sampling. The algorithm takes as parameters: the function, the domain where we want to compute the integral and the available samples. Thus:

1. If the number of points is less than a threshold, then it gives back the estimation of the integral by using the number of points introduced as parameter.
2. Otherwise, it divides the domain in two parts, divides the available points in two and calls, in a recursive manner, the algorithm in each one of the two parts in which the domain has been split.

The third algorithm that has been studied uses regression trees [2]. Given a set of samples $\{(x_t, f(x_t))\}_t$, we aim to obtain a simple function g that approximates f :

$$g(x) = \sum_{i=1}^k a_i \chi_{R_i}(x),$$

where R_i are rectangles that partition the domain of f . Then the following density function is taken:

$$q_g(x) = \frac{|g(x)|}{\int_R |g(z)| dz} = \frac{1}{\sum_{i=1}^k |a_i| V_{R_i}} \sum_{i=1}^k |a_i| \chi_{R_i}(x).$$

Following the density q_g , a new set of samples is generated. These will be used to estimate the integral (the estimation is carried out by using the *Generalized sample mean* where $X \sim q_g$).

The distribution induced by q_g is simple, and in order to generate a sample following distribution one just needs to select a rectangle with probability proportional to $|a_i| V_{R_i}$ and then to take a sample following a distribution $U(int(R_i))$. Broadly speaking, the next steps are followed:

1. A set of initial samples is generated following a uniform distribution.
2. The tree practices several times with the aim of refining g .
3. By using $X \sim q_g$, the *Generalized sample mean* is applied.

The main goal of the method is to improve the estimation of the integral by reducing the variance through the sacrifice of a set of samples, which are the ones employed to obtain the function g . Therefore, the accuracy of the method relies heavily upon the approximation given by the tree; in fact, we can face the following situations:

- The approximation is good. Consequently, a significant variance reduction takes place, the estimation of the integral is accurate and the estimated error is small.
- The approximation is not good enough. Consequently, a variance reduction does not take place, the estimation of the integral is inaccurate and the estimated error is big.

The fourth and last algorithm that has been studied is known as ***StackMonteCarlo***. It is in fact a post-processing method (since it works on the basis of a set of samples generated following a specific distribution $X \sim \omega$), it exploits the variance reduction method control variables and uses a k -fold cross-validation and supervised learning. Therefore, given a set of data generated following ω , this method describes a way of obtaining g and estimating α . In Example 4.6, *StackMonteCarlo* and *Sample mean* are compared.

Finally, in Section 4.5 we compare the methods: VEGAS, MISER, regression tree and *Sample mean*. To this end, we use four functions defined in the rectangle $[0, 1]^d$: the first one is easy to integrate, the second one is not a square-integrable function (the fact that a function is not square-integrable implies that we cannot apply the Central Limit Theorem to estimate the error), the third one is a product of exponential functions, and the fourth one is a sum of trigonometric functions. We draw the graphs for the real absolute error and the estimated absolute error, and obtain some conclusions.

It is noteworthy to mention that in Annex 6 one can find tools of probability and statistics which we will use throughout this work. There is also an Annex 7 with *python* code which implements:

- Monte Carlo method for the calculation of volumes, developed in Chapter 1.
- *Sample mean*, developed in Chapter 2.
- *Hit and miss*, developed in Chapter 2.
- Stratified sampling, developed in Chapter 3.
- Integration with regression trees, developed in Chapter 4.

- *StackMonteCarlo* with regression trees, developed in Chapter 4.

The methods which are not implemented are VEGAS (it can be downloaded in [6]) and MISER (it can be downloaded in [15]).

All the implemented methods in Annex 7 give, at least, the integral or volume and an estimation of the error with a confidence level of 99 %.

The most important python libraries used are: *math* [21], *numpy* [19], *sklearn* [16] (for the regression trees and the k -fold cross-validation) and *matplotlib* [20] for the graphs.

Índice general

Resumen	II
Abstract	VI
1 Cálculo de áreas y volúmenes	1
1.1 Cálculo de áreas	1
1.2 Cálculo de volúmenes	5
1.3 Cálculo de volúmenes en la práctica	7
2 Integración básica	10
2.1 <i>Acierto o fallo</i>	12
2.2 <i>Media en el muestreo</i>	15
2.3 Comparación de ambos métodos	18
3 Métodos de reducción de varianza	21
3.1 <i>Media en el muestreo generalizado</i>	21
3.2 Optimizando la <i>Importancia en el muestreo</i>	23
3.3 Muestreo Estratificado	24
3.4 <i>Variables de control</i>	28
4 Algoritmos de integración	31
4.1 VEGAS	31
4.1.1 VEGAS en una dimensión	31
4.1.2 VEGAS en varias dimensiones	33
4.2 MISER	36
4.3 Árbol de regresión	38
4.4 <i>StackMonteCarlo</i>	42
4.5 Comparación de métodos	46
5 Conclusiones	53
6 Anexo A : Teoría de la Probabilidad	54
7 Anexo B : Código en python generado	59
7.1 Clase Árbol	59
7.2 Métodos de integración implementados	62

1 | Cálculo de áreas y volúmenes

En este capítulo se introducirá el método de Monte Carlo para abordar el cálculo de volúmenes de objetos d dimensionales. En la primera sección, estudiaremos la variante más sencilla del método de Monte Carlo y nos restringiremos al cálculo de áreas (es decir, $d = 2$). Posteriormente, consideraremos una extensión del método anterior para el cálculo de volúmenes de objetos de dimensión arbitraria. Como se observará, el incremento de dimensión del espacio sobre el que estamos trabajando no supondrá gran empeoramiento del método. Esto último motiva el uso del método de Monte Carlo en problemas de volúmenes. En la última sección, se proporcionará una cota práctica general del error para volúmenes.

Los contenidos de este capítulo se han basado en las notas de clase [9].

1.1. Cálculo de áreas

Sea $R \subset \mathbb{R}^2$ un rectángulo, ($R = [a_1, b_1] \times [a_2, b_2]$) y nos proponemos calcular el área de un objeto $D \subseteq R$. Para ello, vamos definir dos variables aleatorias independientes (X, Y) cuyas distribuciones son uniformes; concretamente $X \sim U((a_1, b_1))$ e $Y \sim U((a_2, b_2))$. Gráficamente:

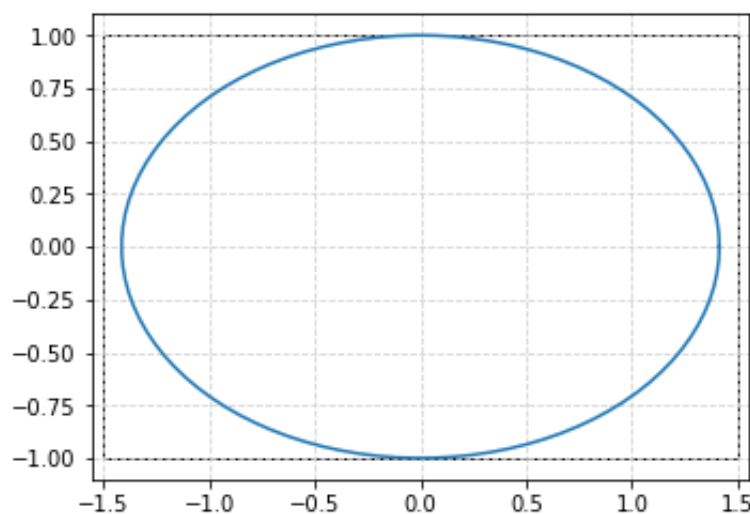


Figura 1.1: Gráfico de una elipse (azul) dentro de un rectángulo

Ahora definiremos una variable aleatoria de Bernoulli $B \sim b(\theta)$ tal que:

$$B = \begin{cases} 1 & \text{si } (X, Y) \in D \\ 0 & \text{si } (X, Y) \notin D \end{cases} \quad (1.1)$$

El parámetro θ es desconocido, pero se puede calcular fácilmente de la siguiente manera:

$$\theta = E[B] = P((X, Y) \in D) = \iint_D f(x, y) dx dy = \frac{A_D}{A_R}, \quad (1.2)$$

donde $f(x, y)$ denota la función de densidad de (X, Y) . Por consiguiente, si determinamos θ , tendremos el área del objeto D , pues $A_D = \theta \times A_R$ y $A_R = (b_1 - a_1) \times (b_2 - a_2)$.

En suma, tenemos $B \sim b(\theta)$ y queremos estimar θ . Para abordar este problema, aplicaremos inferencia estadística sobre la variable aleatoria B . Aplicando la Proposición 6.10 se sigue inmediatamente lo siguiente:

Proposición 1.1:

Sea $B \sim b(\theta)$ la variable aleatoria definida en (1.1) y $(B_1, \dots, B_n)$ una m.a.s. de B entonces:

- $E[\bar{B}] = E[B] = \theta$
- $Var[\bar{B}] = \frac{Var[B]}{n} = \frac{\theta(1-\theta)}{n}$
- $\bar{B} \rightarrow_P E[B] = \theta$

Como veremos más adelante, las dos primeras propiedades tomarán un papel importante a la hora de estimar el error del método. Por otro lado, la tercera propiedad nos dice lo siguiente:

$$\forall \epsilon > 0, \lim_n P(|\bar{B} - \theta| < \epsilon) = 1.$$

Esto se traduce en que si tomamos un error $E > 0$ tan pequeño como queramos y un $\gamma \in (0, 1)$ arbitrariamente pequeño, existirá un $n_{E\gamma}$ tal que si $n \geq n_{E\gamma}$ se tendrá $P(|\bar{B} - \theta| < E) > 1 - \gamma$. Fijado E y γ , tomamos n que verifica la condición anterior y llamamos $\bar{B}_{exp}$ a la media muestral de nuestro conjunto de muestras $(b_1, \dots, b_n)$, entonces se tiene que $|\bar{B}_{exp} - \theta| < E$ con una confianza mayor que $1 - \gamma$.

La convergencia en probabilidad de B nos aporta mucha información a nivel teórico sin embargo, a efectos prácticos, no nos sirve de mucho, pues aún no sabemos el orden de convergencia.

A continuación, se presenta un resultado general que será muy utilizado en todo el desarrollo del trabajo:

Proposición 1.2:

Sea $X_1, \dots, X_n, \dots$ una sucesión de variables aleatorias, independientes e idénticamente distribuidas ($X_i \sim X$) de una distribución con media $E[X]$ y varianza $0 < Var[X] < \infty$. Fijemos un $\gamma \in (0, 1)$, entonces existe un z_γ tal que $\lim_n P\left(|\bar{X} - E[X]| \leq z_\gamma \sqrt{Var[\bar{X}]}\right) = \gamma$. Además el z_γ tiene la siguiente expresión:

$$z_\gamma = \Phi^{-1}\left(\frac{\gamma + 1}{2}\right),$$

donde Φ es la función de distribución de una $N(0, 1)$.

Demostración. Sea $V_n = \frac{\bar{X} - E[X]}{\sqrt{Var[\bar{X}]}}$. Por el teorema central del límite $V_n \rightarrow_D Z \sim N(0, 1)$. La convergencia en distribución quiere decir que $F_{V_n} \rightarrow \Phi$ débilmente; siendo F_{V_n} la función de distribución de V_n y Φ la función de distribución de una normal.

La convergencia débil se traduce en que para todo z , punto de continuidad de Φ (como se trata de una normal, $Cont(\Phi) = \mathbb{R}$) $\lim_n F_{V_n}(z) = \Phi(z)$, además, la convergencia débil implica que $\lim_n F_{V_n}(z-) = \Phi(z)$ (véase Anexo 6).

Teniendo lo anterior en mente, se da la siguiente cadena de igualdades:

$$P(|\bar{X} - E[X]| \leq z\sqrt{Var[\bar{X}]}) = P(-z \leq \frac{\bar{X} - E[X]}{\sqrt{Var[\bar{X}]}} \leq z) = P(-z \leq V_n \leq z) = F_{V_n}(z) - F_{V_n}((-z)-).$$

Tomando $n \rightarrow \infty$, $F_{V_n}(z) - F_{V_n}((-z)-) \rightarrow \Phi(z) - \Phi(-z) = \Phi(z) - (1 - \Phi(z)) = 2\Phi(z) - 1$.

Ahora consideramos la ecuación $2\Phi(z) - 1 = \gamma$ que tiene solución pues Φ es estrictamente monótona y continua en todo $\mathbb{R}$ y viene dada por $z_\gamma = \Phi^{-1}(\frac{\gamma+1}{2})$. En consecuencia, hemos encontrado un z_γ que $\lim_n P(|\bar{X} - E[X]| \leq z_\gamma\sqrt{Var[\bar{X}]}) = \gamma$. $\square$

Ahora resta aplicar el resultado anterior a nuestra variable aleatoria B .

Teorema 1.3:

Sea $B \sim b(\theta)$ la variable aleatoria definida en (1.1) y $(B_1, \dots, B_n)$ una m.a.s. de B entonces:

- $\forall \gamma \in (0, 1)$, existe un z_γ tal que $\lim_n P\left(|\bar{B} - \theta| \leq z_\gamma \sqrt{\frac{\theta(1-\theta)}{n}}\right) = \gamma$.
- **(Error de Monte Carlo para el cálculo de áreas)** Sea la variable aleatoria $A = A_R \bar{B}$, entonces $\forall \gamma \in (0, 1)$, existe un z_γ tal que

$$\lim_n P\left(|A - A_D| \leq z_\gamma A_R^{\frac{1}{2}} \sqrt{A_D \left(1 - \frac{A_D}{A_R}\right) n^{-\frac{1}{2}}}\right) = \gamma.$$

Demostración. Para el primer ítem basta sustituir las fórmulas de los dos primeros puntos de la Proposición 1.1 en el límite de la Proposición 1.2.

Para el segundo, basta multiplicar la expresión obtenida en el primer ítem por el área del rectángulo teniendo en cuenta que $\theta = \frac{A_D}{A_R}$. $\square$

Observación 1.4:

Con lo estudiado hasta ahora, podemos concluir lo siguiente:

- El estadístico que verdaderamente nos da el área es A no $\bar{B}$, pues B nos da una estimación de θ .
- El método es convergente en probabilidad: $|A - A_D| \rightarrow 0$ si $n \rightarrow \infty$.
- El método tiene un orden de convergencia de $n^{-\frac{1}{2}}$. Lo cual es muy lento si lo comparamos con otros métodos deterministas de cálculo de áreas.
- Si A_D es parecido a A_R (lo que se consigue si el rectángulo es más pequeño), entonces el error disminuye.

- Cuando tomamos un A_{exp} ligado a un conjunto de muestras, podría ocurrir que no se verifique la cota $|A_{exp} - A_D| \leq z_\gamma A_R^{\frac{1}{2}} \sqrt{A_D(1 - \frac{A_D}{A_R})} n^{-\frac{1}{2}}$. Dada una confianza $\gamma \in (0, 1)$, existe un z_γ tal que si tomamos un n suficientemente grande, $P\left(|A - A_D| \leq z_\gamma A_R^{\frac{1}{2}} \sqrt{A_D(1 - \frac{A_D}{A_R})} n^{-\frac{1}{2}}\right) \approx \gamma$. Es decir, para una A_{exp} ligada a un conjunto de muestras se verificará la cota $|A_{exp} - A_D| \leq z_\gamma A_R^{\frac{1}{2}} \sqrt{A_D(1 - \frac{A_D}{A_R})} n^{-\frac{1}{2}}$ con una confianza $\approx \gamma$.

Ejemplo 1.5:

Nos planteamos calcular el área de una elipse utilizando el método de Monte Carlo. Por tanto, buscamos el área de $D_{a,b} = \{(x, y) \in \mathbb{R}^2 \mid \frac{x^2}{a^2} + \frac{y^2}{b^2} \leq 1\} \subset R_1 \times R_2$ con $a > 0$ y $b > 0$. Se realizarán los siguientes pasos:

- Definimos $X \sim U(R_1)$ e $Y \sim U(R_2)$ independientes, obteniendo así (X, Y) .
- Definimos B como en (1.1).
- Tomamos una muestra $(b_1, \dots, b_n)$, esto es, tomamos aleatoriamente (siguiendo una distribución uniforme) n muestras dentro de $R_1 \times R_2$ y para cada uno de los pares (x_i, y_i) generamos b_i que tomará el valor 0 si cae fuera de la elipse y el valor 1 si cae dentro.
- Calculamos la media muestral experimental $(\bar{B}_{exp})$, esto es, la media de los b_i y multiplicamos el resultado por el área del rectángulo (generando así A_{exp}).
- El área calculada por el método de Monte Carlo se corresponde con A_{exp} .

Así pues, tomando $a = \sqrt{2}$, $b = 1$, un $n = 1000$, $R_1 = [-3/2, 3/2]$ y $R_2 = [-1, 1]$ obtenemos la siguiente gráfica:

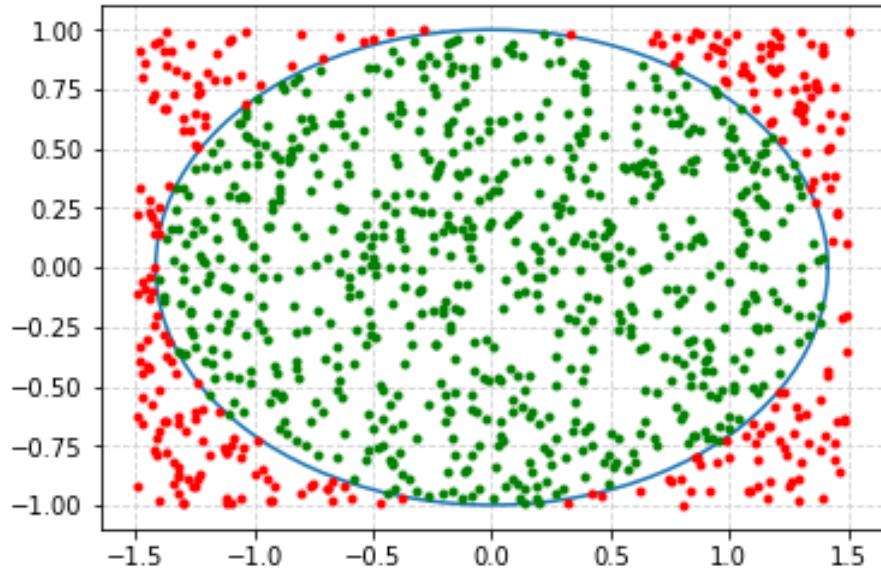


Figura 1.2: Gráfico 1.1 añadiendo las muestras.

Los puntos verdes se corresponden lo las muestras b_i cuyo valor es 1, mientras que, las muestras en rojo, se corresponden con las que su valor es 0. La integral según el método vale $A_{exp} = 4,314$. Tomando una confianza $\gamma = 0,99$, si nos fijamos en el enunciado de la Proposición 1.2, $z_\gamma = \Phi(\frac{\gamma+1}{2})$, por tanto $z_\gamma \approx 2,58$ y podemos calcular $z_\gamma A_R^{\frac{1}{2}} \sqrt{A_D(1 - \frac{A_D}{A_R})} n^{\frac{-1}{2}}$ (en este caso sabemos A_D pero lo normal es que este valor se desconozca). $A_D = a \times b \times \pi$, $A_R = 6$ luego $z_\gamma A_R^{\frac{1}{2}} \sqrt{A_D(1 - \frac{A_D}{A_R})} n^{\frac{-1}{2}} \approx 0,215$ y $|A_{exp} - A_D| = 0,128$, que es menor que 0,215. La cota se cumple.

1.2. Cálculo de volúmenes

El cálculo de áreas se puede extender fácilmente al cálculo de volúmenes en d dimensiones. Siguiendo la misma línea que la subsección anterior, sea $R \subset \mathbb{R}^d$ un rectángulo, ($R = [a_1, b_1] \times \dots \times [a_d, b_d]$) y nos proponemos calcular el volumen de un objeto $D \subseteq R$. Para ello, vamos a definir una variable aleatoria X d -dimensional que sigue una distribución uniforme $X \sim U(int(R))$, donde $int(R)$ denota el interior de R .

Observación 1.6:

Nótese que $X \sim U(int(R))$ es equivalente a definir $X = (X_1, \dots, X_d)$ donde $X_i \sim U((a_i, b_i))$ independientes.

Ahora definiremos una variable aleatoria de Bernoulli $B \sim b(\theta)$ tal que:

$$B = \begin{cases} 1 & \text{si } X \in D \\ 0 & \text{si } X \notin D \end{cases} \quad (1.3)$$

El parámetro θ es desconocido, pero se puede calcular fácilmente de la siguiente manera:

$$\theta = E[B] = P(X \in D) = \int \dots \int_D f(x_1, \dots, x_d) dx_1 \dots dx_d = \frac{V_D}{V_R}, \quad (1.4)$$

donde $f(x_1, \dots, x_d)$ denota la función de densidad de X . Por consiguiente, si determinamos θ , tendremos el volumen del objeto D , pues $V_D = \theta \times V_R$ y $V_R = (b_1 - a_1) \times \dots \times (b_d - a_d)$.

Como se hizo en el cálculo de áreas, haremos inferencia sobre B :

Proposición 1.7:

Sea $B \sim b(\theta)$ la variable aleatoria definida en (1.3) y $(B_1, \dots, B_n)$ una m.a.s. de B entonces:

- $E[\bar{B}] = E[B] = \theta$
- $Var[\bar{B}] = \frac{Var[B]}{n} = \frac{\theta(1-\theta)}{n}$
- $\bar{B} \rightarrow_P E[X] = \theta$

Teorema 1.8:

Sea $B \sim b(\theta)$ la variable aleatoria definida en (1.3) y $(B_1, \dots, B_n)$ una m.a.s. de B entonces:

- $\forall \gamma \in (0, 1)$, existe un z_γ tal que $\lim_n P\left(|\bar{B} - \theta| \leq z_\gamma \sqrt{\frac{\theta(1-\theta)}{n}}\right) = \gamma$.
- **(Error de Monte Carlo para el cálculo de volúmenes I)** Sea la variable aleatoria $V = V_R \bar{B}$, entonces $\forall \gamma \in (0, 1)$, existe un z_γ tal que

$$\lim_n P\left(|V - V_D| \leq z_\gamma V_R^{\frac{1}{2}} \sqrt{V_D \left(1 - \frac{V_D}{V_R}\right) n^{-\frac{1}{2}}}\right) = \gamma.$$

Ejemplo 1.9:

Nos planteamos calcular el volumen de un elipsoide utilizando el método de Monte Carlo. Por tanto, buscamos el volumen de $D_{a,b,c} = \{(x, y, z) \in \mathbb{R}^3 \mid \frac{x^2}{a^2} + \frac{y^2}{b^2} + \frac{z^2}{c^2} \leq 1\} \subset [-a, a] \times [-b, b] \times [-c, c]$ con $a > 0$, $b > 0$ y $c > 0$. Se realizarán los siguientes pasos:

- Definimos $X \sim U((-a, a))$, $Y \sim U((-b, b))$ y $Z \sim U((-c, c))$ independientes, obteniendo así (X, Y, Z) .
- Definimos B como en (1.3).
- Tomamos una muestra $(b_1, \dots, b_n)$.
- Calculamos la media muestral experimental $(\bar{B}_{exp})$, esto es, la media de los b_i y multiplicamos el resultado por el volumen del rectángulo (generando así V_{exp}).
- El volumen calculado por el método de Monte Carlo se corresponde con V_{exp} .

Tomando $n = 1000$, $a = 1$, $b = 2$ y $c = 3$ gráficamente, tenemos lo siguiente:

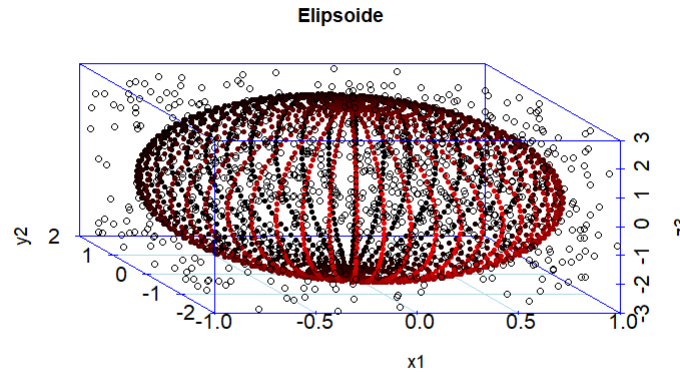


Figura 1.3: Gráfico del Método de Monte Carlo aplicado a un elipsoide (los puntos blancos son las muestras).

La integral según el método vale $V_{exp} = 26,352$. Tomando una confianza $\gamma = 0,99$, $z_\gamma \approx 2,58$ y podemos calcular $z_\gamma V_R^{\frac{1}{2}} \sqrt{V_D(1 - \frac{V_D}{V_R}) n^{-\frac{1}{2}}}$ (en este caso sabemos V_D pero lo normal es que este

valor se desconozca). $V_D = \frac{4}{3} \times a \times b \times c \times \pi$, $V_R = 2a \times 2b \times 2c$ luego $z_\gamma V_R^{\frac{1}{2}} \sqrt{V_D(1 - \frac{V_D}{V_R})} n^{-\frac{1}{2}} \approx 1,949$ y $|V_{exp} - V_D| = 1,219$ que es menor que 1,949. La cota se cumple.

Observación 1.10:

Se puede concluir los mismos items que en el caso del cálculo de áreas. Cabe destacar que el orden de convergencia no depende de la dimensión.

En lo que resta de sección, sólo consideraremos el método de Monte Carlo en volúmenes ya que, como se puede observar, el cálculo de áreas es un caso particular con $d = 2$.

1.3. Cálculo de volúmenes en la práctica

En esta subsección se introducirá una serie de cálculos y resultados que serán útiles desde un punto de vista práctico. De ahora en adelante (salvo en resultados generales), $R = [a_1, b_1] \times \dots \times [a_d, b_d]$, $D \subseteq R$ (el objeto del cual queremos saber el volumen), $X \sim U(int(R))$ y B definida en (1.3).

Ahora nuestro objetivo es dar una alternativa al error descrito en el Teorema 1.8. Ahí, el error viene dado en función de θ que es un parámetro desconocido, pues se desconoce V_D . Y como el θ presente proviene de la varianza, lo que en realidad buscamos es un estimador de la varianza. Un estimador natural de la varianza es el siguiente:

Definición 1.11:

Sea X una variable aleatoria y $(X_1, \dots, X_n)$ una m.a.s. de X se define la varianza muestral como sigue:

$$s^2 = \frac{\sum_{i=1}^n (X_i - \bar{X})^2}{n} = \frac{\sum_{i=1}^n X_i^2}{n} - (\bar{X})^2$$

Proposición 1.12:

Sea $X_1, \dots, X_n, \dots$ una sucesión de variables aleatorias, independientes e idénticamente distribuidas ($X_i \sim X$) de una distribución con media $E[X]$ y varianza $0 < Var[X] < \infty$. Fijemos un $\gamma \in (0, 1)$, entonces existe un z_γ tal que $\lim_n P(|\bar{X} - E[X]| \leq z_\gamma \cdot s \cdot n^{-\frac{1}{2}}) = \gamma$

Demostración. Sea $V_n = \frac{\bar{X} - E[X]}{s/\sqrt{n}}$. Por la Proposición 6.15 $V_n \rightarrow_D Z \sim N(0, 1)$. La convergencia en distribución quiere decir que $F_{V_n} \rightarrow \Phi$ débilmente; siendo F_{V_n} la función de distribución de V_n y Φ la función de distribución de una normal.

La convergencia débil se traduce en que para todo z , punto de continuidad de Φ (como se trata de una normal, $Cont(\Phi) = \mathbb{R}$) $\lim_n F_{V_n}(z) = \Phi(z)$, además, la convergencia débil implica que $\lim_n F_{V_n}(z-) = \Phi(z)$.

Teniendo lo anterior en mente, se da la siguiente cadena de igualdades:

$$P(|\bar{X} - E[X]| \leq z \cdot s \cdot n^{-1/2}) = P(-z \leq \frac{\bar{X} - E[X]}{s/\sqrt{n}} \leq z) = P(-z \leq V_n \leq z) = F_{V_n}(z) - F_{V_n}((-z)-).$$

Tomando $n \rightarrow \infty$, $F_{V_n}(z) - F_{V_n}((-z)) \rightarrow \Phi(z) - \Phi(-z) = \Phi(z) - (1 - \Phi(z)) = 2\Phi(z) - 1$.

Ahora consideramos la ecuación $2\Phi(z) - 1 = \gamma$ que tiene solución pues Φ es estrictamente monótona y continua en todo $\mathbb{R}$ y viene dada por $z_\gamma = \Phi^{-1}(\frac{\gamma+1}{2})$. En consecuencia, hemos encontrado un z_γ que $\lim_n P(|\bar{X} - E[X]| \leq z_\gamma \cdot s \cdot n^{-1/2}) = \gamma$. $\square$

Teorema 1.13:

Sea $B \sim b(\theta)$ la variable aleatoria definida en (1.3), $(B_1, \dots, B_n)$ una m.a.s. de B y $s^2 = \frac{\sum_{i=1}^n (B_i - \bar{B})^2}{n}$ la varianza muestral de B entonces:

- $\forall \gamma \in (0, 1)$, existe un z_γ tal que $\lim_n P(|\bar{B} - \theta| \leq z_\gamma \cdot s \cdot n^{-\frac{1}{2}}) = \gamma$.
- **(Error de Monte Carlo para el cálculo de volúmenes II)** Sea la variable aleatoria $V = V_R \bar{B}$, entonces $\forall \gamma \in (0, 1)$, existe un z_γ tal que

$$\lim_n P(|V - V_D| \leq z_\gamma \cdot s \cdot V_R \cdot n^{-\frac{1}{2}}) = \gamma.$$

Observación 1.14:

El segundo ítem del teorema anterior nos viene a decir lo siguiente: dada una confianza $\gamma \in (0, 1)$, existe un z_γ tal que si tomamos un n suficientemente grande, $P(|V - V_D| \leq z_\gamma \cdot s \cdot V_R \cdot n^{-\frac{1}{2}}) \approx \gamma$. Es decir, para una V_{exp} y una s_{exp} ligada a un conjunto de muestras se verificará la cota $|V_{exp} - V_D| \leq z_\gamma \cdot s_{exp} \cdot V_R \cdot n^{-\frac{1}{2}}$ con una confianza $\approx \gamma$. La cantidad dada por $z_\gamma \cdot s_{exp} \cdot V_R \cdot n^{-\frac{1}{2}}$ es conocida en la práctica (la cota proporcionada en el Teorema 1.8 no es conocida).

Ejemplo 1.15:

En las mismas circunstancias que el Ejemplo 1.9, con las mismas muestras estimamos la varianza de la variable aleatoria con la varianza muestral obteniendo: $s = 0,498$, de modo que $z_\gamma \cdot s \cdot V_R \cdot n^{-\frac{1}{2}} \approx 1,95$.

Terminamos este capítulo con un ejemplo de cálculos de volúmenes en 4 dimensiones.

Ejemplo 1.16:

Consideremos la esfera 4-dimensional de radio 2, es decir, $D = \{(x, y, z, t) \in \mathbb{R}^4 : x^2 + y^2 + z^2 + t^2 \leq 4\} \subset [-2, 2]^4$. Seguimos el mismo procedimiento que en el Ejemplo 1.9 pero ahora tenemos una variable aleatoria 4-dimensional: (X_1, X_2, X_3, X_4) con $X_i \sim U((-2, 2))$. Como en los ejemplos anteriores, tomemos $n = 1000$ y $z_\gamma = 2,58$. Se obtiene lo siguiente:

- $V_{exp} = 76,288$ y $V_D = 78,957$
- $|V_{exp} - V_D| = 2,669$
- $s_{exp} = 0,457$
- $z_\gamma \cdot s \cdot V_R \cdot n^{-\frac{1}{2}} \approx 9,545$

Efectivamente, $|V_{exp} - V_D| = 2,669 < z_\gamma \cdot s \cdot V_R \cdot n^{-\frac{1}{2}} \approx 9,545$.

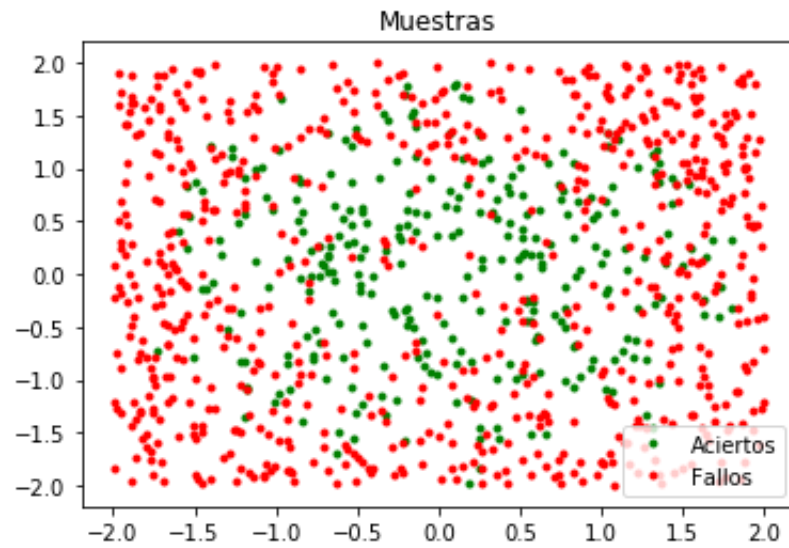


Figura 1.4: Proyección de las muestras sobre las dos primeras coordenadas.

2 | Integración básica

En este capítulo se introducirá dos técnicas de integración. En primer lugar se explicará el método *Acierto o fallo*, muy intuitivo basado en el cálculo de volúmenes (el cálculo de una integral de funciones no negativas no es más que el cálculo del volumen bajo la curva). Posteriormente, se introducirá el método *Media en el muestreo* que como probaremos, será mejor que *Acierto o fallo*. Los contenidos de este capítulo están basados en las notas de clase [9] y en los libros [3, 8].

Antes de pasar a la primera técnica, presentaremos el problema de calcular la integral de manera formal:

Sea $f : R \subset \mathbb{R}^d \rightarrow \mathbb{R}$ una función cuyo dominio de definición es un rectángulo $R = [a_1, b_1] \times \dots \times [a_d, b_d]$ integrable Lebesgue sobre ese dominio. Se busca calcular:

$$I_f = \int_R f(x) dx. \quad (2.1)$$

Definimos los método de integración de Monte Carlo de la siguiente forma:

Definición 2.1:

Sea f una función que cumple las condiciones anteriores. Un **método de Monte Carlo para integración** (MI) es una variable aleatoria M tal que $E[M] = I_f$. Sea $(M_1, \dots, M_n)$ una m.a.s. de M , se llama **estimador de la integral asociado a M** (EMI) a la media muestral de M , $\bar{M}$.

Dado un método de Monte Carlo para integración y su respectivo estimador asociado, podemos aplicarle la Proposición 6.10:

Proposición 2.2:

Sea M un MI, $(M_1, \dots, M_n)$ una m.a.s. de M y $\bar{M}$ su EMI asociado, entonces:

- $E[\bar{M}] = E[M] = I_f$
- $Var[\bar{M}] = \frac{Var[M]}{n}$
- $\bar{M} \rightarrow_P E[M] = I_f$

Como consecuencia de la Proposición 1.2 se tiene que:

Proposición 2.3:

Sea M un MI, $(M_1, \dots, M_n)$ una m.a.s. de M y $\bar{M}$ su EMI asociado. Supongamos $0 <$

$Var[M] < \infty$ y fijemos un $\gamma \in (0, 1)$, entonces existe un z_γ tal que

$$\lim_n P \left(|\bar{M} - I_f| \leq z_\gamma \sqrt{Var[\bar{M}]} \right) = \gamma.$$

Observación 2.4:

El límite de la Proposición 2.3 se puede reescribir de la siguiente forma:

$$\lim_n P \left(|\bar{M} - I_f| \leq z_\gamma \sqrt{Var[M] \cdot n^{-1/2}} \right) = \gamma,$$

pues $Var[\bar{M}] = \frac{Var[M]}{n}$. Como la dimensión del dominio de f es arbitrario, el orden de convergencia no depende de la dimensión.

La proposición anterior no contempla el caso en que $Var[M] = 0$:

Observación 2.5:

Si $Var[M] = 0$, las propiedades de las variables aleatorias nos dicen que $M = E[M] = I_f$ casi seguramente, esto es, $P(M = I_f) = 1$. Si se diera este caso, en la práctica solo necesitaríamos tomar un tamaño muestral $n = 1$ y el valor de la muestra sería la integral buscada con probabilidad 1.

Como ocurre en el caso de volúmenes, $Var[M]$ nos es desconocida; por consiguiente, en la práctica, necesitamos un estimador. El siguiente resultado se sigue de la Proposición 6.15:

Proposición 2.6:

Sea M un MI, $(M_1, \dots, M_n)$ una m.a.s. de M , $\bar{M}$ su EMI asociado y s_M^2 su varianza muestral. Supongamos $0 < Var[M] < \infty$ y fijemos un $\gamma \in (0, 1)$, entonces existe un z_γ tal que $\lim_n P(|\bar{M} - I_f| \leq z_\gamma \cdot s_M \cdot n^{\frac{-1}{2}}) = \gamma$.

La Proposición 2.3 justifica la siguiente definición:

Definición 2.7:

Sea M un MI, $(M_1, \dots, M_n)$ una m.a.s. de M y $\bar{M}$ su EMI asociado; consideremos otro MI, M' , $(M'_1, \dots, M'_n)$ una m.a.s. de M' y $\bar{M}'$ su EMI asociado entonces se dice que M es más **exacto** que M' si $Var[\bar{M}] \leq Var[\bar{M}']$.

Como se vio en la demostración de la Proposición 1.2, dado un γ , el valor de z_γ no depende de la variable aleatoria considerada (en concreto, $z_\gamma = \Phi^{-1}(\frac{\gamma+1}{2})$) luego si M es más exacto que M' entonces se tiene que:

$$z_\gamma \sqrt{Var[\bar{M}]} \leq z_\gamma \sqrt{Var[\bar{M}']} \text{ que es equivalente a } \sqrt{Var[M]} \leq \sqrt{Var[M']},$$

lo que indica que la cota del error para M es menor que la cota de M' .

2.1. Acierto o fallo

Sea $f : R \subset \mathbb{R}^d \rightarrow \mathbb{R}$ una función cuyo dominio de definición es un rectángulo $R = [a_1, b_1] \times \dots \times [a_d, b_d]$ medible Lebesgue sobre ese dominio, $|f(x)| \leq c$ (con c una constante) e I_f la integral de f en R . Sea $f^+(x) = \max\{f(x), 0\}$ y $f^-(x) = \max\{-f(x), 0\}$. Gráficamente:

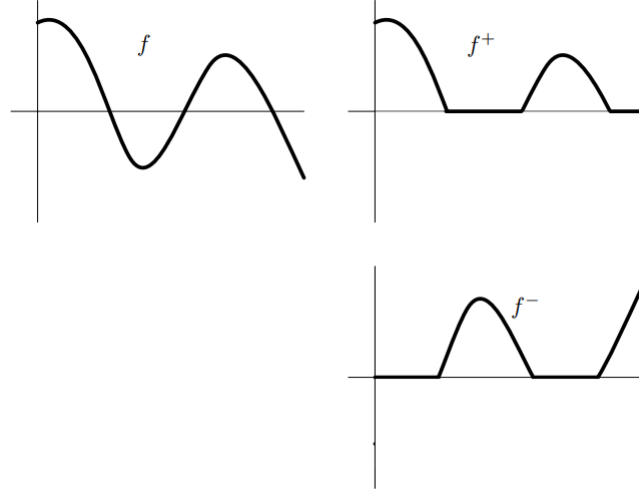


Figura 2.1: Gráfico de la parte positiva y negativa de una función.

Como $|f(x)| \leq c$, podemos considerar el grafo del valor absoluto de la función dentro de un rectángulo de dimensión $d + 1$: $\text{Graf}(|f|) = \{(x, y) \in R \times [0, c] \mid y = |f(x)|\} \subset R \times [0, c]$ (pues $|f| = f^+ + f^-$). Sea X una variable aleatoria tal que $X \sim U(\text{int}(R) \times (0, c))$, definiremos H de la siguiente manera:

$$H = \begin{cases} 1 & \text{si } X \in D^+ \\ -1 & \text{si } X \in D^- \\ 0 & \text{en otro caso} \end{cases} \quad (2.2)$$

con los conjuntos D^+ y D^- definidos:

$$D^+ = \{(x, y) \in R \times (0, c) \mid f^+(x) > 0, y \leq f^+(x)\} \quad (2.3)$$

$$D^- = \{(x, y) \in R \times (0, c) \mid f^-(x) > 0, y \leq f^-(x)\} \quad (2.4)$$

calculemos, pues, la media y varianza de H :

Proposición 2.8:

Sea H la variable aleatoria definida en (2.2) entonces se verifica lo siguiente:

- $E[H] = \frac{1}{V} \cdot I_f$ donde $V = V_R \times c$, volumen del rectángulo en el cuál está encerrado el grafo de f .
- $\text{Var}[H] = \frac{1}{V^2} \left(V \int_R |f(x)| dx - I_f^2 \right)$

Demostración. Teniendo en cuenta que $f = f^+ - f^-$ se sigue que:

$$\begin{aligned} E[H] &= P(X \in D^+) - P(X \in D^-) = \int_{D^+} \frac{1}{V} dx - \int_{D^-} \frac{1}{V} dx \\ &= \frac{1}{V} (V_{D^+} - V_{D^-}) = \frac{1}{V} \left(\int_R f^+(x) dx - \int_R f^-(x) dx \right) = \frac{1}{V} I_f. \end{aligned}$$

Para el segundo ítem, aplicando un razonamiento similar al anterior y teniendo en cuenta que $|f| = f^+ + f^-$ se sigue que:

$$E[H^2] = P(X \in D^+) + P(X \in D^-) = \frac{1}{V} \int_R |f(x)| dx.$$

Por tanto, tenemos que:

$$Var[H] = E[H^2] - E[H]^2 = \frac{1}{V^2} \left(V \int_R |f(x)| dx - I_f^2 \right).$$

□

Consideremos la variable aleatoria $M = V \cdot H$ donde V es el volumen del rectángulo (una constante) donde está encerrada la gráfica del valor absoluto de f y H la variable aleatoria definida en (2.2).

Definición 2.9:

M así definida es un MI y dada una m.a.s. $(M_1, \dots, M_n)$ de M , $\bar{M}$ es su EMI asociado. Dicha variable aleatoria es lo que se conoce como el método de **Acierto o fallo**.

Observación 2.10:

De la definición de M , es directo que $E[M] = I_f$ y $Var[M] = V \int_R |f(x)| dx - I_f^2$.

Ejemplo 2.11:

Nos proponemos calcular la integral $\int_{-2}^2 (x^2 - 1) dx$. Como $|x^2 - 1| \leq 3$ entonces $Graf(|f|) \subset [-2, 2] \times [0, 3]$; $c = 3$ y $R = [-2, 2]$. Así pues realizamos el siguiente procedimiento:

- Definimos $X_1 \sim U((-2, 2))$ y $X_2 \sim U((-0, 3))$ independientes, obteniendo así $X := (X_1, X_2)$.
- Definimos H como en (2.2).
- Tomamos una muestra $(h_1, \dots, h_n)$.
- Calculamos la media muestral experimental $(\bar{H}_{exp})$, esto es, la media de los h_i y multiplicamos el resultado por el volumen del rectángulo (generando así $\bar{M}_{exp} = V \bar{H}_{exp}$).
- La integral buscada se corresponde con $\bar{M}_{exp}$.

Tomando $n = 100$, gráficamente, tenemos lo siguiente:

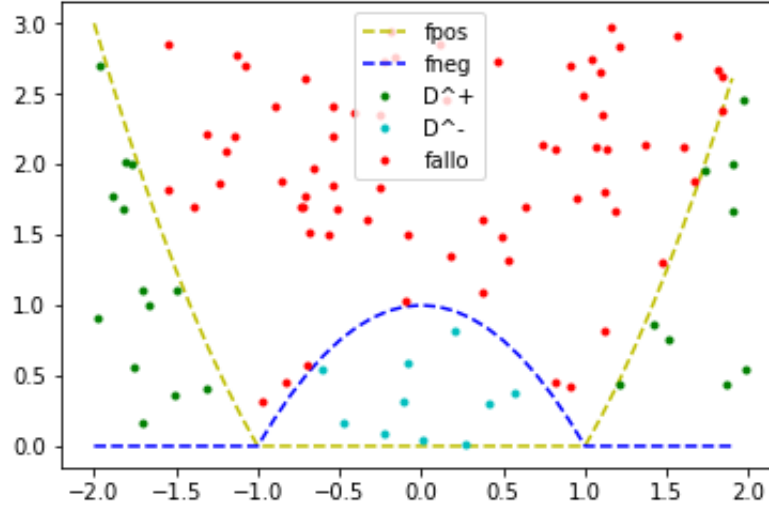


Figura 2.2: Aplicación del método *Acerto o fallo*

Fijado $z_\gamma = 2,58$, obtenemos los siguientes resultados:

- $\bar{M}_{exp} = 1,44$ y $I_f = \frac{4}{3} \approx 1,333$
- $|\bar{M}_{exp} - I_f| = 0,1019$
- $s_{M,exp} = V \cdot 0,553 = 12 \cdot 0,553 = 6,636$
- Tomamos la expresión de la Proposición 2.6: $z_\gamma \cdot s_M \cdot n^{-\frac{1}{2}} \approx 1,712$

Efectivamente $|\bar{M}_{exp} - I_f| = 0,1019 \leq z_\gamma \cdot s_M \cdot n^{-\frac{1}{2}} \approx 1,712$. En este caso el error es pequeño pero la estimación del mismo con un 99% de confianza es grande; esto quiere decir que si repetimos el experimento seguramente obtengamos un valor de $\bar{M}_{exp}$ que diste mucho de I_f . Esto se soluciona tomando un n más grande.

Observación 2.12:

Dado un error δ y una confianza γ buscamos un tamaño muestral n adecuado tal que $P(|\bar{M} - I_f| \leq \delta) \approx (\geq) \gamma$. Para ello utilizamos el teorema central del límite del siguiente modo:

$$P(|\bar{M} - I_f| \leq \delta) = P(|\bar{M} - I_f| / (\text{Var}[\bar{M}]) \leq \delta / \text{Var}[\bar{M}]) \approx_{TCL} P\left(|Z| \leq \frac{\delta}{\sqrt{\text{Var}[\bar{M}]}}\right) = 2P\left(Z \leq \frac{\delta}{\sqrt{\text{Var}[\bar{M}]}}\right) - 1.$$

El símbolo $\approx_{TCL}$ indica que aplicamos el Teorema Central del Límite para un número de muestras suficientemente grande, siendo $Z \sim N(0, 1)$.

Por otro lado, se tiene que $\text{Var}[\bar{M}] = \frac{\text{Var}[M]}{n}$ y $\text{Var}[M] \leq V \int_R |f(x)| dx \leq V^2$. Por consiguiente, $P\left(Z \leq \frac{\delta}{\sqrt{\text{Var}[\bar{M}]}}\right) \geq P\left(Z \geq \frac{\delta\sqrt{n}}{V}\right)$. En consecuencia:

$$P(|\bar{M} - I_f| \leq \delta) \approx 2P\left(Z \leq \frac{\delta}{\sqrt{\text{Var}[\bar{M}]}}\right) - 1 \geq 2P\left(Z \leq \frac{\delta\sqrt{n}}{V}\right) - 1.$$

Así, tomando $n \geq \left(\frac{\Phi^{-1}(\frac{\gamma+1}{2}) \cdot V}{\delta}\right)^2$, se obtiene el n buscado.

Ejemplo 2.13:

Vamos a aplicar la observación anterior al Ejemplo 2.11. Queremos un error de $\delta = 0,1$, con una confianza del 99 %; así pues, tomamos un número de muestras $n = \left(\frac{12 \cdot 2,58}{0,1}\right)^2 \approx 96000$. Obtenemos los siguientes resultados:

- $\bar{M}_{exp} = 1,312$ y $I_f = \frac{4}{3} \approx 1,333$
- $|\bar{M}_{exp} - I_f| = 0,021$
- $s_{M,exp} = V \cdot 0,553 = 12 \cdot 0,567 = 6,804$
- Tomamos la expresión de la Proposición 2.6: $z_\gamma \cdot s_M \cdot n^{-\frac{1}{2}} \approx 0,057$

Efectivamente se tiene que $|\bar{M}_{exp} - I_f| = 0,021 \leq z_\gamma \cdot s_M \cdot n^{-\frac{1}{2}} \approx 0,057 < \delta$

2.2. Media en el muestreo

Una vez estudiado el método *Acierto o fallo*, introduciremos el método de integración de Monte Carlo *media en el muestreo* que, como demostraremos, será más exacto (en el sentido de la Definición 2.7). Sea $f : R \subset \mathbb{R}^d \rightarrow \mathbb{R}$ una función cuyo dominio de definición es un rectángulo $R = [a_1, b_1] \times \dots \times [a_d, b_d]$ y $f \in L^2(R)$. Sea $X \sim U(\text{int}(R))$, definamos $Y = f(X)$. Esta variable aleatoria tiene las siguientes propiedades:

Proposición 2.14:

La esperanza y varianza de la variable aleatoria Y vienen dadas por:

- $E[Y] = \frac{1}{V_R} \cdot I_f$
- $\text{Var}[Y] = \frac{1}{V_R} \int_R \left(f(x) - \frac{I_f}{V_R}\right)^2 dx = \frac{1}{V_R} \int_R f^2(x) dx - \frac{I_f^2}{V_R^2}$

donde V_R es el volumen de R donde está definida la función f .

Demostración. Ambos items se siguen fácilmente de la definición de esperanza, varianza y de la fórmula $\text{Var}[Y] = E[Y^2] - E[Y]^2$. □

Por tanto, si consideramos la variable aleatoria $S = V_R \cdot Y$ donde V_R es el volumen del rectángulo (una constante), dominio de definición de $f(x)$.

Observación 2.15:

De la definición de S , es directo que $E[S] = I_f$ y $Var[S] = V_R \cdot \int_R (f(x) - \frac{I_f}{V_R})^2 dx$.

Definición 2.16:

S así definida es un MI y dada una m.a.s. $(S_1, \dots, S_n)$ de S , $\bar{S}$ es su EMI asociado. Dicha variable aleatoria es lo que se conoce como el método de **media en el muestreo**.

Ejemplo 2.17:

Nos proponemos calcular la integral $\int_0^\pi \int_0^\pi \int_0^\pi \sin(x) \sin(y) \sin(z) dx dy dz$. Realizamos el siguiente procedimiento:

- Definimos $X \sim U((0, \pi)^3)$.
- Definimos $Y = f(X)$.
- Tomemos la muestra $(y_1, \dots, y_n)$.
- Calculamos la media muestral experimental $(\bar{Y}_{exp})$, esto es, la media de los y_i y multiplicamos el resultado por el volumen del rectángulo (generando así $\bar{S}_{exp} = V\bar{Y}_{exp}$).
- La integral buscada se corresponde con $\bar{S}_{exp}$.

Tomando $n = 100$ y $z_\gamma = 2,58$, hacemos el experimento y obtenemos:

- $\bar{S}_{exp} = 6,451$ y $I_f = 8$
- $|\bar{S}_{exp} - I_f| = 1,549$
- $s_{S,exp} = 7,071$
- Tomamos la expresión de la Proposición 2.6: $z_\gamma \cdot s_S \cdot n^{-\frac{1}{2}} \approx 1,824$

Ahora vamos a hacer lo mismo que en *Acierto o fallo* (Observación 2.12): dado un error δ y una confianza γ nos planteamos encontrar un n adecuado. Para ello necesitamos previamente probar un pequeño lema:

Lema 2.18:

Sea X una variable aleatoria tal que $M = \sup X$ y $m = \inf X$ entonces $Var[X] \leq \frac{(M-m)^2}{4}$.

Demostración. Definimos la función $f(t) = E[(X - t)^2]$. Desarrollando la expresión y simplificando se llega a que:

$$f(t) = E[((X - E[X]) - (t - E[X]))^2] = E[(X - E[X])^2] + (t - E[X])^2 \geq E[(X - E[X])^2]$$

y se alcanza el mínimo si $t = E[X]$. Además, se tiene que:

$$f\left(\frac{M+m}{2}\right) = \frac{1}{4}E[((X - m) + (X - M))^2].$$

Como $X - m \geq 0$ y $X - M \leq 0$ entonces:

$$((X - m) + (X - M))^2 \leq ((X - m) - (X - M))^2 = (M - m)^2.$$

Tomando esperanzas:

$$\frac{1}{4}E \left[((X - m) + (X - M))^2 \right] \leq \frac{1}{4}E \left[((X - m) - (X - M))^2 \right] = \frac{(M - m)^2}{4}.$$

Y podemos concluir que:

$$Var[X] = f(E[X]) \leq f\left(\frac{M+m}{2}\right) \leq \frac{(M-m)^2}{4}.$$

□

Observación 2.19:

Supongamos $m \leq f(x) \leq M$ para todo $x \in R$. Dado un error δ y una confianza γ buscamos un tamaño muestral n adecuado tal que $P(|\bar{S} - I_f| \leq \delta) \approx (\geq) \gamma$. Para ello utilizamos el teorema central del límite del siguiente modo:

$$\begin{aligned} P(|\bar{S} - I_f| \leq \delta) &= P(|\bar{S} - I_f| / \sqrt{Var[\bar{S}]} \leq \delta / \sqrt{Var[\bar{S}]}) \approx_{TCL} P\left(|Z| \leq \frac{\delta}{\sqrt{Var[\bar{S}]}}\right) \\ &= 2P\left(Z \leq \frac{\delta}{\sqrt{Var[\bar{S}]}}\right) - 1. \end{aligned}$$

Se tiene que $Var[\bar{S}] = \frac{Var[S]}{n}$ y, aplicando el lema anterior se tiene que $Var[S] \leq \frac{V_R^2(M-m)^2}{4}$ (pues $S = V_R \cdot f(X)$). Por consiguiente, $P\left(Z \leq \frac{\delta}{\sqrt{Var[\bar{S}]}}\right) \geq P\left(Z \leq \frac{2\delta\sqrt{n}}{V_R(M-m)}\right)$. En consecuencia:

$$P(|\bar{S} - I_f| \leq \delta) \approx 2P\left(Z \leq \frac{\delta}{\sqrt{Var[\bar{S}]}}\right) - 1 \geq 2P\left(Z \leq \frac{2\delta\sqrt{n}}{V_R(M-m)}\right) - 1 = \gamma.$$

Así, tomando $n \geq \left(\frac{\Phi^{-1}(\frac{\gamma+1}{2}) \cdot V_R(M-m)}{2\delta}\right)^2$, se obtiene el n buscado.

Ejemplo 2.20:

Aplicamos la observación anterior al Ejemplo 2.17. Queremos obtener la integral con una confianza del 99 % y un error $\delta = 0,1$. Es obvio que $-1 \leq f(x, y, z) = \sin(x) \sin(y) \sin(z) \leq 1$; entonces, el n que tomaremos será:

$$n = \left(\frac{2,58\pi^3(1 - (-1))}{2 \cdot 0,5}\right)^2 \approx 640000$$

Llevamos a cabo el experimento y obtenemos lo siguiente:

- $S_{exp} = 8,008$ y $I_f = 8$
- $|M_{exp} - I_f| = 0,008$
- $s_{S,exp} = 7,499$

- Tomamos la expresión de la Proposición 2.6: $z_\gamma \cdot s_S \cdot n^{\frac{-1}{2}} \approx 0,024$

2.3. Comparación de ambos métodos

Ya hemos visto dos métodos de integración de Monte Carlo (*Acierto o fallo* y *Media en el muestreo*) ahora nos planteamos la siguiente pregunta: ¿Cuál de ellos es mejor (en el sentido de la Definición 2.7)? La respuesta la tiene el siguiente teorema:

Teorema 2.21:

Sea $f : R \subset \mathbb{R}^d \rightarrow \mathbb{R}$ una función cuyo dominio de definición es un rectángulo $R = [a_1, b_1] \times \dots \times [a_d, b_d]$ medible Lebesgue sobre ese dominio, $|f(x)| \leq c$ (con c una constante) e I_f la integral de f en R . Consideremos S el MI *Media en el muestreo* con $(S_1, \dots, S_n)$ una m.a.s. de S y $\bar{S}$ es su EMI asociado; M el MI *Acierto o fallo* con $(M_1, \dots, M_n)$ una m.a.s. de M y $\bar{M}$ es su EMI asociado. Entonces S es más exacto que M .

Demostración. En primer lugar hay que notar que $Var[\bar{S}] = V_R^2 \cdot Var[\bar{Y}] = \frac{V_R^2}{n} \cdot Var[Y]$, donde Y es la variable aleatoria definida en la sección anterior. Así pues, se tiene que:

$$\begin{aligned} Var[\bar{S}] &= \frac{V_R^2}{n} \left(\int_R \frac{|f(x)|^2}{V_R} dx - \frac{1}{V_R^2} \left(\int_R f(x) dx \right)^2 \right) \\ &= \frac{1}{n} \left(V_R \int_R |f(x)|^2 dx - \left(\int_R f(x) dx \right)^2 \right) \\ &\leq \frac{1}{n} \left(V_R \cdot c \int_R |f(x)| dx - \left(\int_R f(x) dx \right)^2 \right) \\ &= \frac{1}{n} Var[M] = Var[\bar{M}]. \end{aligned}$$

□

La conclusión de este teorema es clara, el método de *Media en el muestreo* es superior a *Acierto o fallo*. A continuación comparamos experimentalmente ambos métodos:

Ejemplo 2.22:

Vamos a calcular la integral $\int_{-2}^2 (x^2 - 1) dx$ utilizando los dos métodos. Fijaremos $z_\gamma = 2,58$ y variaremos el número de muestras.

n	Acierto o fallo				Media en el muestreo			
	$\bar{H}_{exp}$	e_{real}	s_H^2	e_{cota}	$\bar{S}_{exp}$	e_{real}	s_S^2	e_{cota}
50	0.96	0.37	39.38	2.29	1.21	0.12	27.14	1.90
100	1.44	0.11	44	1.71	1.67	0.33	21.51	1.20
150	1.36	0.03	45.19	1.42	1.18	0.15	22.8	1.00
200	1.62	0.29	45.62	1.23	1.60	0.27	25.91	0.93
500	1.25	0.09	47.98	0.79	1.60	0.27	23.84	0.56
1000	1.44	0.11	49.19	0.57	1.61	0.28	23.08	0.39
5000	1.32	0.02	46.20	0.24	1.34	0.01	22.54	0.17
50000	1.30	0.03	46.31	0.08	1.32	0.01	22.60	0.05

En esta tabla $|I_f - \bar{X}_{exp}| = e_{real}$ y $z_\gamma \cdot s_X \cdot n^{-\frac{1}{2}} = e_{cota}$. Nos fijamos en e_{cota} que se corresponde con el error previsto. Se observa claramente que es menor si se emplea *Media en el muestreo* esto es así ya que su varianza estimada es menor ($s_H^2 < s_M^2$ en todos los experimentos).

Ejemplo 2.23:

Vamos a comparar los dos métodos aplicándolos a función del Ejemplo 2.17. Variando el número de muestras tal que así:

$$n \in \{1000, 2000, 3000, 4000, 5000, 10000, 15000, 20000, 25000, 30000\}$$

Claramente, *Media en el muestreo* (SM) ofrece una estimación del error absoluto menor en todos los casos.

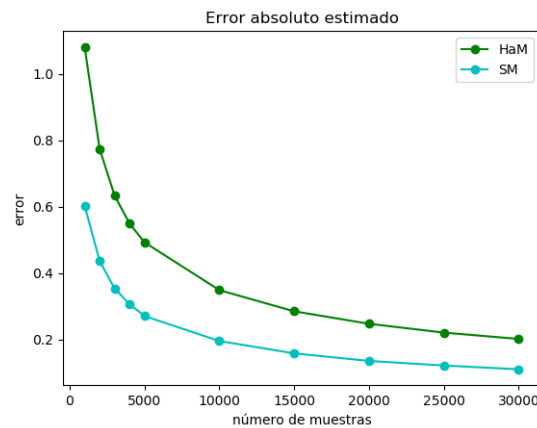


Figura 2.3: Error absoluto estimado.

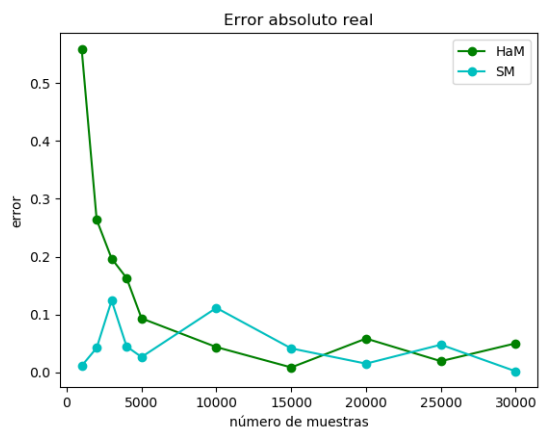


Figura 2.4: Error absoluto estimado.

3 | Métodos de reducción de varianza

Los ejemplos de aplicación de los métodos vistos en los dos capítulos anteriores ponen de manifiesto que el error real y la estimación del error absoluto son bastante malos salvo que se aumente el número de muestras. Esto motiva la necesidad de desarrollar refinamientos de los métodos de Monte Carlo básicos.

Una conclusión del capítulo anterior es que el método *Media en el muestreo* es más exacto que *Acierto o fallo*. Así pues, vamos a intentar mejorar *Media en el muestreo* introduciendo tres técnicas. Pero antes de comenzar a explicar la primera de ellas vamos a "generalizar" *Media en el muestreo*.

Este capítulo está basado en el libro [3] y en el artículo [12].

3.1. *Media en el muestreo generalizado*

Sea $f : R \subset \mathbb{R}^d \rightarrow \mathbb{R}$ una función cuyo dominio de definición es un rectángulo $R = [a_1, b_1] \times \dots \times [a_d, b_d]$ integrable Lebesgue sobre ese dominio con $\int_R f^2(x) dx < \infty$. Sea $X \sim \omega$, donde ω es una función de densidad (X es una v.a continua) tal que $\text{supp}(f) \subset \text{supp}(\omega)$ con $\int_R \frac{f^2(x)}{\omega(x)} dx < \infty$. Entonces se tiene que:

$$I_f = \int_R f(x) dx = \int_R \frac{f(x)}{\omega(x)} \omega(x) dx = E \left[\frac{f(X)}{\omega(X)} \right]. \quad (3.1)$$

Observación 3.1:

$\text{supp}(f)$ se refiere al soporte de la función f , es decir:

$$\text{supp}(f) = \overline{\{x \in \mathbb{R}^d : f(x) \neq 0\}}$$

Observación 3.2:

Nótese que, el método *Media en el muestreo* explicado en el capítulo anterior, se obtiene tomando $\omega(x) = \frac{1}{V_R}$ lo que equivale a decir que $X \sim U(\text{int}(R))$.

Definamos entonces la variable aleatoria $Y = \frac{f(X)}{\omega(X)}$. Es inmediato ver que Y es un MI.

Definición 3.3:

La variable aleatoria Y así definida es un MI y, dada una m.a.s. $(Y_1, \dots, Y_n)$ de Y , $\bar{Y}$ es su EMI asociado. Dicha variable aleatoria se conoce como el método de **Media en el muestreo generalizado**. La función de densidad ω se le denomina *importancia en el muestreo*.

Establecemos lo que vale la varianza de Y y la media en esta proposición.

Proposición 3.4:

La esperanza y varianza de la variable aleatoria Y vienen dadas por:

- $E[Y] = I_f$
- $Var[Y] = \int_R \left(\frac{f(x)}{\omega(x)} - I_f \right)^2 dx = \int_R \frac{f^2(x)}{\omega(x)} dx - I_f^2$

Demostración. El primer ítem se desarrolla en (3.1) y el segundo ítem se sigue fácilmente de la definición de varianza y de la fórmula $Var[Y] = E[Y^2] - E[Y]^2$. $\square$

Ejemplo 3.5:

Supongamos que queremos calcular la integral $\int_{-3}^3 (x^2 - 10) dx$. Vamos a escoger un *importancia en el muestreo* $\omega \sim N(2, 1)$, es decir sigue una normal de $\mu = 2$ y $\sigma^2 = 1$. Realizamos el siguiente procedimiento:

- Definimos $X \sim N(2, 1)$.
- Definimos $Y = \frac{f(X)}{\omega(X)}$, donde ω es la función de densidad de una $N(2, 1)$.
- Tomemos la muestra $(y_1, \dots, y_n)$.
- Calculamos la media muestral experimental ($\bar{Y}_{exp}$), esto es, la media de los y_i .
- La integral buscada se corresponde con $\bar{Y}_{exp}$.

Tomando $n = 1000$ y $z_\gamma = 2,58$, hacemos el experimento y obtenemos:

- $\bar{Y}_{exp} = -39,042$ y $I_f = -42$
- $|\bar{Y}_{exp} - I_f| = 2,957$
- $s_{Y,exp} = 347,089$
- Tomamos la expresión de la Proposición 2.6: $z_\gamma \cdot s_{Y,exp} \cdot n^{-\frac{1}{2}} \approx 28,317$

En cambio, si tomamos $\omega \sim U((-3, 3))$. Se tienen los siguientes resultados con el mismo n y z_γ :

- $\bar{Y}_{exp} = -41,225$ y $I_f = -42$
- $|\bar{Y}_{exp} - I_f| = 0,774$
- $s_{Y,exp} = 16,542$

- Tomamos la expresión de la Proposición 2.6: $z_\gamma \cdot s_{Y,exp} \cdot n^{\frac{-1}{2}} \approx 1,349$

Este ejemplo nos indica que hay que tener cuidado en la elección de la *importancia en el muestreo*

3.2. Optimizando la *Importancia en el muestreo*

Recordemos que según la Definición 2.7 un MI es más exacto que otro si su varianza es menor. A raíz de esto y atendiendo a la definición de *Media en el muestreo generalizado* nos preguntamos: ¿qué ω es la mejor?, en otras palabras, ¿qué ω es la que minimiza la varianza de Y ? La respuesta a esta cuestión es respondida por el siguiente teorema:

Teorema 3.6:

Sea $f : R \subset \mathbb{R}^d \rightarrow \mathbb{R}$ una función cuyo dominio de definición es un rectángulo $R = [a_1, b_1] \times \dots \times [a_d, b_d]$ con $f \in L^2(R)$. Supongamos $f \neq 0$. Entonces, la función de densidad:

$$q(x) = \frac{|f(x)|}{\int_R |f(z)| dz}$$

minimiza la varianza, esto es $Var_q[Y] \leq Var_\omega[Y]$ ($Var_\omega[Y] = Var[Y]$, con $Y = \frac{f(X)}{\omega(X)}$ y $X \sim \omega$), para toda función de densidad ω tal que $supp(f) \subset supp(\omega)$ con $\int_R \frac{f^2(x)}{\omega(x)} dx < \infty$. Además la varianza mínima viene dada por:

$$\left(\int_R |f(x)| dx \right)^2 - \left(\int_R f(x) dx \right)^2.$$

En consecuencia, si la función $f(x)$ es de signo constante en R , $Var_q[Y] = 0$.

Demostración. Afirmo que:

$$Var_\omega[Y] = \left(\int_R |f(x)| dx \right)^2 - \left(\int_R f(x) dx \right)^2 + \left(\int_R |f(x)| dx \right)^2 \cdot E_\omega \left[\left(\frac{q(X) - \omega(X)}{\omega(X)} \right)^2 \right],$$

donde E_ω indica que $X \sim \omega$. Para simplificar, llamemos $A := \int_R |f(x)| dx$. Así pues desarrollando E_ω :

$$\begin{aligned} E_\omega \left[\left(\frac{q(X) - \omega(X)}{\omega(X)} \right)^2 \right] \\ = \int_R \frac{\frac{|f(x)|^2}{A^2} - 2\frac{|f(x)|}{A}\omega(x) + \omega(x)^2}{\omega(x) dx}. \end{aligned}$$

De esta manera se tiene que:

$$A^2 E_\omega \left[\left(\frac{q(X) - \omega(X)}{\omega(X)} \right)^2 \right] = \int_R \frac{|f(x)|^2}{\omega(x)} dx - \int_R 2|f(x)|A dx + \int_R A^2 \omega(x) dx = \int_R \frac{f^2(x) dx}{\omega(x)} - A^2.$$

Por la Proposición 3.4 se tiene que:

$$Var_{\omega}[Y] = \int_R \frac{f^2(x)}{\omega(x)} dx - I_f^2 = \int_R \frac{f^2(x)}{\omega(x)} dx - A^2 + A^2 - I_f^2 = A^2 E_{\omega} \left[\left(\frac{q(X) - \omega(X)}{\omega(X)} \right)^2 \right] + A^2 - I_f^2.$$

Por otro lado, se verifica que:

$$\begin{aligned} Var_{\omega}[Y] &= \left(\int_R |f(x)| dx \right)^2 - \left(\int_R f(x) dx \right)^2 + \left(\int_R |f(x)| dx \right)^2 \cdot E_{\omega} \left[\left(\frac{q(X) - \omega(X)}{\omega(X)} \right)^2 \right] \\ &= A^2 - I_f^2 + A^2 E_{\omega} \left[\left(\frac{q(X) - \omega(X)}{\omega(X)} \right)^2 \right] \\ &\geq \left(\int_R |f(x)| dx \right)^2 - \left(\int_R f(x) dx \right)^2. \end{aligned}$$

Y se alcanza el mínimo si y solo si $\omega = q$. □

3.3. Muestreo Estratificado

Supongamos que nuestra *importancia en el muestreo* tiene la siguiente forma:

$$\omega(x) = \sum_{i=1}^l p_i \omega_i(x),$$

donde $\sum_{i=1}^l p_i = 1$ con los $p_i \geq 0$ y las ω_i son funciones que verifican:

- ω_i es una función de densidad para todo $i = 1, \dots, l$.
- $supp(\omega) = \bigcup_{i=1}^l supp(\omega_i)$.
- $supp(\omega_i) \cap supp(\omega_j) = \emptyset$, para todo $i \neq j$.

De estas condiciones se deduce que $p_i = \int_{supp(\omega_i)} \omega(x) dx$, para todo $i = 1 \dots l$, lo cual asegura que ω es una función de densidad.

Definición 3.7:

Los subconjuntos $supp(\omega_i)$ se denominan *estratos*.

La idea detrás de este método de reducción de varianza consiste en calcular $\int_{supp(\omega_i)} f(x) dx$ empleando el MI $Y_i = \frac{f(X_i)}{\omega_i(X_i)}$ donde $X_i \sim \omega_i$ y $\bar{Y}_{n_i}$ es su EMI asociado. Como $supp(f) \subset supp(\omega) \subset R$ entonces se tiene que:

$$I_f = \int_R f(x) dx = \int_{\bigcup_{i=1}^l supp(\omega_i)} f(x) dx = \sum_{i=1}^l \int_{supp(\omega_i)} f(x) dx. \quad (3.2)$$

Como estimamos cada $\int_{supp(\omega_i)} f(x) dx$ con $\bar{Y}_{n_i}$ (de manera independiente) entonces la integral en R estimará con:

$$\bar{Y}_{st} = \sum_{i=1}^l \bar{Y}_{n_i}.$$

Definición 3.8:

Dado el siguiente conjunto de funciones de densidad $\{\omega_i\}_{i=1}^l$ que verifican:

- $\text{supp}(\omega_i) \cap \text{supp}(\omega_j) = \emptyset$
- $\text{supp}(f) \subset \bigcup_{i=1}^l \text{supp}(\omega_i) \subset R$

y dado el siguiente conjunto de enteros positivos $\{n_i\}_{i=1}^l$, la variable aleatoria $\bar{Y}_{st} = \sum_{i=1}^l \bar{Y}_{n_i}$ se denomina **estimador de muestreo estratificado**.

Observación 3.9:

Un estimador de muestreo estratificado, $\bar{Y}_{st}$ viene determinado por $\{\omega_i\}_{i=1}^l$ y por $\{n_i\}_{i=1}^l$. Dados $\{p_i\}_{i=1}^l$ reales tales que $\sum_{i=1}^l p_i = 1$ y $p_i \geq 0$ podemos construir la función de densidad $\omega(x) = \sum_{i=1}^l p_i \omega_i(x)$. Esta ω tiene asociada el estimador $\bar{Y}_{st}$. Luego hay infinitas funciones de densidad que tienen asociadas el mismo estimador de muestreo estratificado.

Ese estimador tiene las siguientes propiedades:

Proposición 3.10:

Sea $\bar{Y}_{st}$ el estimador de muestreo estratificado asociado a $\{\omega_i(x)\}_{i=1}^l$ con un número de muestras para cada estratos de $n_1, \dots, n_l$. Entonces se tiene lo siguiente:

- $E[\bar{Y}_{st}] = I_f$
- $Var[\bar{Y}_{st}] = \sum_{i=1}^l \frac{1}{n_i} \left[\int_{\text{supp}(\omega_i)} \frac{f^2(x)}{\omega_i(x)} dx - \left(\int_{\text{supp}(\omega_i)} f(x) dx \right)^2 \right]$

Demostración. La esperanza de la suma de variables aleatorias es la suma de las esperanzas, y cada $E[\bar{Y}_{n_i}] = \int_{\text{supp}(\omega_i)} f(x) dx$. El resultado se sigue de (3.2).

Respecto a la varianza, aplicando la Proposición 3.4 y teniendo en cuenta que las variables aleatorias $\bar{Y}_{n_i}$ son independientes, se tiene que:

$$Var[\bar{Y}_{st}] = \sum_{i=1}^l Var[\bar{Y}_{n_i}] = \sum_{i=1}^l \frac{1}{n_i} Var[Y_i] = \sum_{i=1}^l \frac{1}{n_i} \left[\int_{\text{supp}(\omega_i)} \frac{f^2(x)}{\omega_i(x)} dx - \left(\int_{\text{supp}(\omega_i)} f(x) dx \right)^2 \right].$$

□

La variable aleatoria $\bar{Y}_{st}$ cumple estas dos propiedades relacionadas con la convergencia:

Proposición 3.11:

En las mismas condiciones que la proposición anterior. La variable aleatoria $\bar{Y}_{st}$ verifica lo siguiente:

- $\bar{Y}_{st} \rightarrow_P I_f$, cuando $\min\{n_1, \dots, n_l\} \rightarrow \infty$
- $\frac{\bar{Y}_{st} - I_f}{\sqrt{Var[\bar{Y}_{st}]}} \rightarrow_D Z$, cuando $\min\{n_1, \dots, n_l\} \rightarrow \infty$, siendo $Z \sim N(0, 1)$.

Demostración. Véase [3, Sección 6.4.2]. □

La varianza de $\bar{Y}_{st}$ nos es desconocida en la práctica. Así pues necesitamos un estimador.

Proposición 3.12:

En las mismas condiciones que la proposición anterior. Consideremos estimador dado por $s_{st}^2 = \sum_{i=0}^l \frac{s_i^2}{n_i}$ donde s_i^2 es el estimador de la varianza de Y_i (varianza muestral). Entonces se verifica:

$$\frac{\bar{Y}_{st} - I_f}{s_{st}} \rightarrow_D Z$$

cuando $\min\{n_1, \dots, n_l\} \rightarrow \infty$, siendo $Z \sim N(0, 1)$. En consecuencia, para todo $\gamma \in (0, 1)$, existe z_γ tal que:

$$P(|\bar{Y}_{st} - I_f| \leq z_\gamma \cdot s_{st}) \rightarrow \gamma \text{ cuando } \min\{n_1, \dots, n_l\} \rightarrow \infty.$$

Demostración. Véase [3, Sección 6.4.2]. □

Una vez introducido el muestreo estratificado, nos preguntamos cómo escoger los n_i de manera que la varianza sea minimizada. Es decir, establecer el número de muestras n_i que hay que elegir de cada *estrato* de modo que se consiga una varianza menor que escoger $\sum_i n_i$ muestras siguiendo la distribución $X \sim \omega$.

Teorema 3.13:

Sea $\bar{Y}_{st}$ el estimador de muestreo estratificado asociado a $\{\omega_i\}_{i=1}^l$ con un número de muestras para cada *estrato* de $n_1, \dots, n_l$. Dado un tamaño de muestreo n ($\sum_i n_i = n$), la varianza $Var[\bar{Y}_{st}]$ es minimizada tomando:

$$n_i = n \frac{\sqrt{Var[Y_i]}}{\sum_{j=0}^l \sqrt{Var[Y_j]}}$$

y la varianza mínima es

$$\frac{\left(\sum_{j=0}^l \sqrt{Var[Y_j]}\right)^2}{n} \leq Var[\bar{Y}]$$

donde $\bar{Y}$ es el EMI del método *Media en el muestreo generalizado* asociado a ω cualquier función de densidad que tenga asociada el estimador $\bar{Y}_{st}$, es decir, cualquier ω de la forma $\omega(x) = \sum_{i=1}^l p_i \omega_i(x)$ con $p_i \geq 0$ para todo i y $\sum_i p_i = 1$ (véase la Observación 3.9).

Demostración. Denotaremos $\sqrt{Var[Y_i]} = \sigma_i$. Usando la desigualdad de Cauchy-Schwarz se tiene que:

$$\begin{aligned} & \frac{\sum_i \sigma_i}{\sqrt{n}} \\ &= \langle (\sqrt{n_1/n}, \dots, \sqrt{n_l/n}), (\sigma_1/\sqrt{n_1}, \dots, \sigma_l/\sqrt{n_l}) \rangle \\ &\leq \sqrt{\sum_i \frac{\sigma_i^2}{n_i}}. \end{aligned}$$

Por consiguiente,

$$\text{Var}[\bar{Y}_{st}] = \sum_i \frac{\sigma_i^2}{n_i} \geq \frac{(\sum_i \sigma_i)^2}{n}$$

y se da la igualdad si y solo si existe una constante c tal que $\sqrt{n_i/n} = c\sigma_i/\sqrt{n_i}$ para todo i , lo que implica que $n_i = c\sqrt{n}\sigma_i$ para todo i . Sabiendo que $\sum_i n_i = n$ entonces se tiene que $c = \sqrt{n}/\sum_i \sigma_i$. Luego tenemos que se da la igualdad si y solo si:

$$n_i = n \frac{\sigma_i}{\sum_j \sigma_j}.$$

Sea ω una función de densidad que tiene asociada $\bar{Y}_{st}$. Como $Y \sim \omega$, generar un número aleatorio siguiendo esa distribución consiste en generar $j \in \{1, \dots, l\}$ con una probabilidad p_j y luego generar un número siguiendo la distribución ω_j . En consecuencia, por la ley de la varianza (Proposición 6.7), se verifica que:

$$\begin{aligned} \text{Var}[Y] &= E[Y|j] + \text{Var}[Y|j] \\ &= \sum_i \frac{\sigma_i^2}{p_i} + \text{Var}[E[Y|j]] \\ &\geq \sum_i \frac{\sigma_i^2}{p_i} \\ &\geq \left(\sum_{i=1}^l \sigma_i \right)^2. \end{aligned}$$

La última desigualdad se obtiene aplicando de nuevo Cauchy-Schwarz a los vectores $(\sqrt{p_1}, \dots, \sqrt{p_l})$ y $(\sigma_1/\sqrt{p_1}, \dots, \sigma_l/\sqrt{p_l})$. Dividiendo por n se tiene la segunda parte del teorema. $\square$

Observación 3.14:

Los valores n_i descritos en el teorema anterior no son necesariamente enteros. Sin embargo, la idea es escoger enteros lo más cercanos posibles a los valores descritos en dicho teorema.

Este teorema tiene una consecuencia recogida en la siguiente proposición:

Proposición 3.15:

Sea $\bar{Y}_{st}$ el estimador del muestreo estratificado en las mismas condiciones que el teorema anterior. Sea $\omega(x) = \sum_i p_i \omega_i(x)$ una función de densidad asociada al estimador. Para $n = \sum_i n_i$, si tomamos $n_i = np_i$ entonces:

$$\text{Var}[\bar{Y}_{st}] = \frac{1}{n} \sum_{i=1}^l \frac{\sigma_i^2}{p_i} \leq \text{Var}[\bar{Y}],$$

donde $\bar{Y}$ es el EMI del método *Media en el muestreo generalizado* asociado a ω con tamaño muestral n y $\sqrt{\text{Var}[Y_i]} = \sigma_i$.

Demostración. En la demostración del teorema anterior hemos la siguiente desigualdad:

$$Var[\bar{Y}] = \frac{Var[Y]}{n} \geq \frac{1}{n} \sum_i \frac{\sigma_i^2}{p_i}.$$

Por otro lado,

$$Var[\bar{Y}_{st}] = \sum_i \frac{\sigma_i^2}{n_i} = \frac{1}{n} \sum_i \frac{\sigma_i^2}{p_i}$$

□

Ejemplo 3.16:

Supongamos que se quiere calcular la integral $\int_{-6}^6 (x^2 + 9) dx$. Se propone tomar la siguiente función de densidad:

$$\omega(x) = 0,4 \cdot \chi_{(-6,-2)}(x) \frac{1}{4} + 0,2 \cdot \chi_{(-2,2)}(x) \frac{1}{4} + 0,4 \cdot \chi_{(2,6)}(x) \frac{1}{4}$$

Cada ω_i es una uniforme. Es como si dividiéramos el intervalo $(-6, 6)$ y diéramos más *peso* a los intervalos $(-6, -2)$ y $(2, 6)$. Se realizará lo siguiente:

- Se calculan las integrales $\int_{-6}^{-2}$, $\int_{-2}^2$ y $\int_2^6$ utilizando el método de *Media en el muestreo* con un número de muestras $n_i = np_i$.
- Se suman las estimaciones anteriores obteniendo $\bar{Y}_{st,exp}$ que será nuestra estimación de la integral en el intervalo $(-6, 6)$.

Tomando $n = 10000$ y $z_\gamma = 2,58$ obtenemos en un experimento lo siguiente:

- $n_1 = 4000 = n_3$, $n_2 = 2000$. $\bar{Y}_{st,exp} = 252,202$ e $I_f = 252$.
- $|\bar{Y}_{st,exp} - I_f| = 0,202$
- $s_{st,exp}^2 = 0,717$
- Usando Proposición 3.12: $z_\gamma \cdot s_{st} \approx 2,185$

Ahora, si en vez de aplicar muestreo estratificado aplicamos *Media en el muestreo generalizado* con ω definida al principio de este ejemplo, al hacer un experimento, se tiene que:

- $\bar{Y}_{exp} = 252,721$
- $|\bar{Y}_{exp} - I_f| = 0,721$
- $s_{exp}^2 = 0,769$
- $z_\gamma \cdot s_Y \cdot n^{\frac{-1}{2}} \approx 2,264$

3.4. Variables de control

Como en los capítulos anteriores, estamos interesados en calcular la integral:

$$\int_R f(x) dx.$$

Supongamos que disponemos de una función g definida en R que aproxima f razonablemente bien y su integral $\int_R g(x) dx$ sabemos calcularla. Entonces dado $\alpha \in \mathbb{R}$ se verifica lo siguiente:

$$\int_R f(x) dx = \int_R f(x) - \alpha g(x) + \alpha g(x) dx = \alpha \int_R g(x) dx + \int_R \frac{f(x) - \alpha g(x)}{\omega(x)} \omega(x) dx,$$

donde ω es una función de distribución conocida. Así pues se sigue que:

$$\int_R f(x) dx = \alpha I_g + E \left[\frac{f(X) - \alpha g(X)}{\omega(X)} \right].$$

Ahora solo falta ver el valor de α que hace que todo lo anterior tenga sentido y sea útil. Esto viene dado en el siguiente teorema:

Teorema 3.17:

En las condiciones anteriores, si tomamos

$$\alpha = \frac{Cov \left[\frac{f(X)}{\omega(X)}, \frac{g(X)}{\omega(X)} \right]}{Var \left[\frac{g(X)}{\omega(X)} \right]},$$

entonces se minimiza la varianza

$$Var \left[\frac{f(X) - \alpha g(X)}{\omega(X)} \right]$$

y vale

$$Var \left[\frac{f(X)}{\omega(X)} \right] - \frac{\left(Cov \left[\frac{f(X)}{\omega(X)}, \frac{g(X)}{\omega(X)} \right] \right)^2}{Var \left[\frac{g(X)}{\omega(X)} \right]}.$$

En consecuencia, lo anterior es menor que $Var \left[\frac{f(X)}{\omega(X)} \right]$ si la covarianza es no nula.

Demostración. Para simplificar, llamamos:

$$\sigma_f^2 = Var \left[\frac{f(X)}{\omega(X)} \right]$$

$$\sigma_g^2 = Var \left[\frac{g(X)}{\omega(X)} \right]$$

$$\sigma_{f,g} = Cov \left[\frac{f(X)}{\omega(X)}, \frac{g(X)}{\omega(X)} \right]$$

Entonces, teniendo en cuenta que $Var[X + Y] = Var[X] + Var[Y] + 2Cov[X, Y]$, obtenemos una función que depende de α :

$$Var \left[\frac{f(X) - \alpha g(X)}{\omega(X)} \right] = \sigma_f^2 - 2\alpha \sigma_{f,g} + \alpha^2 \sigma_g^2 = h(\alpha).$$

Derivando respecto de α , es fácil ver que h tiene un mínimo en:

$$\hat{\alpha} = \frac{\sigma_{f,g}}{\sigma_g^2}$$

y que, evaluando en $\hat{\alpha}$ se tiene:

$$h(\hat{\alpha}) = \sigma_f^2 - \frac{\sigma_{f,g}^2}{\sigma_g^2} \leq \sigma_f^2$$

siendo la desigualdad estricta si la covarianza no es nula. □

Observación 3.18:

En la práctica lo que se nos presenta es un conjunto de muestras que siguen una distribución $X \sim \omega$ y la función g nos viene dada, entonces para saber α tenemos que estimar de alguna forma $\sigma_{f,g}$. Para ello se suele emplear el siguiente estimador:

$$s_{f,g} = \frac{\sum_{i=1}^n (A_i - \bar{A})(B_i - \bar{B})}{n},$$

con $A_i = \frac{f(X_i)}{\omega(X_i)}$ y $B_i = \frac{g(X_i)}{\omega(X_i)}$.

Observación 3.19:

Intuitivamente α se podría ver como un indicador de cuán útil es la función g para nuestros propósitos, es decir, si $|\alpha| \approx 0$ entonces g no nos será muy útil.

Este método será utilizado en *StackMonteCarlo* explicado en la Sección 4.4.

4 | Algoritmos de integración

En este capítulo vamos a introducir dos algoritmos muy utilizados: VEGAS [5, 6, 7, 10] y MISER [11, 15, 10]. El primero de ellos está basado en el método de reducción de varianza *importancia en el muestreo* óptimo y el segundo está basado en el muestreo estratificado. Una vez introducidos estos algoritmos, emplearemos una técnica muy extendida en el universo de *Machine Learning*: árboles de regresión [2]. Esta técnica de regresión tiene (entre otras) estas dos características:

- El algoritmo que emplea para entrenar el árbol con muestras es simple y voraz.
- Aproxima una función por otra que es simple.

Como veremos, sacaremos ventaja de estas dos características. La última técnica que trataremos en este capítulo (*StackMonteCarlo*[12]) es un método de post-procesamiento que intenta mejorar el método *Media en el muestreo* generalizado dando otra forma de estimar la integral; está basado también en técnicas usadas en *Machine Learning*.

Cabe destacar que, cada uno de los algoritmos que vamos a estudiar, explota de manera directa alguna técnica de reducción de varianza citada en el capítulo anterior.

4.1. VEGAS

VEGAS es un algoritmo que aplica muestreo adaptativo (es decir, un algoritmo que cambia su conducta durante la ejecución atendiendo a las características de la función que integra) y está basado en la técnica de reducción de varianza *importancia en el muestreo* óptimo. Este algoritmo es iterativo y va construyendo una función de densidad $q(x)$ (la cual, como veremos, es sencilla) conforme pasan las iteraciones. Este algoritmo busca calcular la integral de una función en un rectángulo:

$$I_f = \int_R f(x) dx = \int_R \frac{f(x)}{q(x)} q(x) dx = E \left[\frac{f(X)}{q(X)} \right].$$

Empezaremos explicando el funcionamiento del algoritmo en el caso de una variable, posteriormente de dos variables y su generalización a d variables se sigue fácilmente de dos variables.

4.1.1. VEGAS en una dimensión

Se quiere calcular la integral:

$$I = \int_a^b f(x) dx$$

Se particiona el dominio de integración de la siguiente manera:

$$\begin{aligned}
a &= x_0 \\
x_1 &= x_0 + \Delta x_0 \\
x_2 &= x_1 + \Delta x_1 \\
&\dots \\
x_N &= x_{N-1} + \Delta x_{N-1} = b
\end{aligned}$$

La función de densidad que se va a utilizar tiene la siguiente forma:

$$q(x) = \sum_{i=0}^{N-1} \frac{1}{N\Delta x_i} \chi_{[x_i, x_{i+1})}(x) = \sum_{i=0}^{N-1} \frac{1}{J_i} \chi_{[x_i, x_{i+1})}(x), \quad (4.1)$$

con $J_i = N\Delta x_i$. Evidentemente, $\sum_{i=0}^{N-1} \frac{\Delta x_i}{J_i} = 1$. Consideremos entonces la variable aleatoria $Y = \frac{f(X)}{q(X)}$ con $X \sim q(x)$; por la Proposición 3.4 la varianza vale:

$$Var[Y] = \int_a^b \frac{f^2(x)}{q(x)} dx - I_f^2 = \sum_{i=0}^{N-1} J_i \int_{x_i}^{x_{i+1}} f^2(x) dx - I_f^2.$$

Buscamos calcular los valores de J_i que minimizan la varianza anterior, esto es:

$$\begin{aligned}
&\min \sum_{i=0}^{N-1} J_i \int_{x_i}^{x_{i+1}} f^2(x) dx \\
&\text{sujeto a } \sum_{i=0}^{N-1} \frac{\Delta x_i}{J_i} = 1
\end{aligned}$$

Aplicando multiplicadores de Lagrange se sigue que:

$$L(J_0, \dots, J_{N-1}, \lambda) = \sum_{i=0}^{N-1} J_i \int_{x_i}^{x_{i+1}} f^2(x) dx + \lambda \left(\sum_{i=0}^{N-1} \frac{\Delta x_i}{J_i} - 1 \right).$$

Derivando e igualando a 0:

$$\frac{\partial L}{\partial J_i} = \int_{x_i}^{x_{i+1}} f^2(x) dx - \lambda \frac{\Delta x_i}{J_i^2} = 0.$$

Por tanto, despejando λ para cada $i = 0, \dots, N-1$, se sigue que

$$\frac{J_i^2}{\Delta x_i} \int_{x_i}^{x_{i+1}} f^2(x) dx = N^2 \Delta x_i \int_{x_i}^{x_{i+1}} f^2(x) dx \quad (4.2)$$

es una constante para todo $i = 0, \dots, N-1$. VEGAS ajusta la partición hasta que esta última condición es satisfecha (la forma en la que modifica la longitud de los intervalos puede consultarse en [5]); para ello, en cada iteración, se emplea un conjunto de muestras para estimar las integrales:

$$\int_{x_i}^{x_{i+1}} f^2(x) dx.$$

Utiliza esa información para modificar los Δx_i y así obtener una nueva $q(x)$ para la siguiente iteración (en la primera iteración comienza con una partición uniforme, es decir, $\Delta x_i = \Delta x_j$ para

todo $i \neq j$). Intuitivamente, la expresión (4.2) nos dice que los incrementos Δx_i serán más pequeños en zonas donde la función $|f(x)|$ tenga un valor elevado.

En cada iteración U , se generan un conjunto de muestras siguiendo la función de densidad que tenga en esa iteración. Usa ese conjunto para refinar los Δx_i y también para hacer una estimación de $\int_a^b f(x) dx$:

$$I_U = \sum_{j=1}^M \frac{f(x_j)}{q(x_j)}$$

y de la varianza:

$$\sigma_U^2 = \frac{1}{M} \sum_{j=1}^M \left(\frac{f(x_j)}{q(x_j)} - I_U \right)^2.$$

Sea $c_U = \sigma_U / \sqrt{M}$, por la Proposición 2.6 se tiene que, dada una confianza de 99 %, para un M suficientemente grande:

$$|I_U - I_f| \leq 2,58 \cdot c_U.$$

Cuando se ha llegado al número máximo de iteraciones, se calcula la estimación final de la integral utilizando información de cada iteración:

$$\bar{I} = \bar{c}^2 \sum_i \frac{I_i}{c_i^2}$$

con

$$\bar{c} = \sqrt{\frac{1}{\sum_i \frac{1}{c_i^2}}}.$$

Es decir, se hace una media ponderada de cada estimación dando más importancia a las I_i tales que su c_i^2 es menor.

Cabe destacar que, generar una muestra siguiendo la función de densidad (4.1) es sencillo, pues basta con seleccionar un subintervalo al azar y tomar la muestra siguiendo una uniforme en dicho intervalo.

4.1.2. VEGAS en varias dimensiones

Se quiere calcular:

$$I_f = \int_a^b \int_c^d f(x, y) dx dy.$$

El dominio de integración se particiona en rectángulos de manera similar a una dimensión:

$$\begin{aligned} a &= x_0 \\ x_1 &= x_0 + \Delta x_0 \\ x_2 &= x_1 + \Delta x_1 \\ &\dots \\ x_N &= x_{N-1} + \Delta x_{N-1} = b \end{aligned}$$

$$\begin{aligned}
c &= y_0 \\
y_1 &= y_0 + \Delta y_0 \\
y_2 &= y_1 + \Delta y_1 \\
&\dots \\
y_N &= y_{N-1} + \Delta y_{N-1} = d
\end{aligned}$$

Ahora la función de distribución que se va a considerar es la que sigue:

$$q(x, y) = q_x(x)q_y(y)$$

donde:

$$q_x(x) = \sum_{i=0}^{N-1} \frac{1}{N\Delta x_i} \chi_{[x_i, x_{i+1})}(x) = \sum_{i=0}^{N-1} \frac{1}{J_i^x} \chi_{[x_i, x_{i+1})}(x) \quad (4.3)$$

$$q_y(y) = \sum_{i=0}^{N-1} \frac{1}{N\Delta y_i} \chi_{[y_i, y_{i+1})}(y) = \sum_{i=0}^{N-1} \frac{1}{J_i^y} \chi_{[y_i, y_{i+1})}(y) \quad (4.4)$$

con $J_i^x = N\Delta x_i$ y $J_i^y = N\Delta y_i$. Evidentemente, $\sum_{i=0}^{N-1} \frac{\Delta x_i}{J_i^x} = 1$ y $\sum_{i=0}^{N-1} \frac{\Delta y_i}{J_i^y} = 1$. Consideremos entonces la variable aleatorio $Y = \frac{f(X)}{q(X)}$ con $X \sim q(x, y)$; por la Proposición 3.4 la varianza vale:

$$\text{Var}[Y] = \int_a^b \int_c^d \frac{f^2(x, y)}{q_x(x)q_y(y)} dx dy - I_f^2.$$

Vamos a llevar a cabo un procedimiento similar que en caso de una dimensión. Fijando los J_i^y , en otras palabras, fijando $q_y(y)$, la varianza anterior se puede escribir como sigue:

$$\text{Var}[Y] = \sum_{i=0}^{N-1} J_i^x \int_{x_i}^{x_{i+1}} \int_c^d \frac{f^2(x, y)}{q_y(y)} dx dy - I_f^2.$$

Buscamos calcular los valores de J_i^x que minimizan la varianza anterior, esto es:

$$\begin{aligned}
&\text{mín} \sum_{i=0}^{N-1} J_i^x \int_{x_i}^{x_{i+1}} \int_c^d \frac{f^2(x, y)}{q_y(y)} dx dy \\
&\text{sujeto a } \sum_{i=0}^{N-1} \frac{\Delta x_i}{J_i^x} = 1
\end{aligned}$$

Aplicando multiplicadores de Lagrange como en el caso de una dimensión, llegamos a que

$$\frac{(J_i^x)^2}{\Delta x_i} \int_{x_i}^{x_{i+1}} \int_c^d \frac{f^2(x, y)}{q_y(y)} dx dy = N^2 \Delta x_i \int_{x_i}^{x_{i+1}} \int_c^d \frac{f^2(x, y)}{q_y(y)} dx dy$$

es una constante para todo $i = 0, \dots, N-1$. Por simetría se obtiene que

$$\frac{(J_i^y)^2}{\Delta y_i} \int_{y_i}^{y_{i+1}} \int_a^b \frac{f^2(x, y)}{q_x(x)} dy dx = N^2 \Delta y_i \int_{y_i}^{y_{i+1}} \int_a^b \frac{f^2(x, y)}{q_x(x)} dy dx$$

es una cosntante para todo $i = 0, \dots, N-1$.

VEGAS ajusta la partición hasta que estas dos últimas condiciones son satisfechas (la forma en la que modifica la longitud de los intervalos puede consultarse en [5]); para ello, en cada iteración, se emplean un conjunto de muestras para estimar las intergrales:

$$\int_{x_i}^{x_{i+1}} \int_c^d \frac{f^2(x, y)}{q_y(y)} dx dy$$

$$\int_{y_i}^{y_{i+1}} \int_a^b \frac{f^2(x, y)}{q_x(x)} dy dx$$

En cada iteración U , se generan un conjunto de muestras siguiendo la función de densidad que tenga en esa iteración. Usa ese conjunto para refinar los Δx_i y Δy_i ; y también para hacer una estimación de la $\int_a^b \int_c^d f(x, y) dx dy$:

$$I_U = \sum_{j=1}^M \frac{f(x_j, y_j)}{q(x_j, y_j)}$$

y de la varianza

$$\sigma_U^2 = \frac{1}{M} \sum_{j=1}^M \left(\frac{f(x_j, y_j)}{q(x_j, y_j)} - I_U \right)^2.$$

Sea $c_U = \sigma_U / \sqrt{M}$, por la Proposición 2.6 se tiene que, dada una confianza de 99 %, para un M suficientemente grande:

$$|I_U - I_f| \leq 2,58 \cdot c_U.$$

Cuando se ha llegado al número máximo de iteraciones, se calcula la estimación final de la integral utilizando información de cada iteración:

$$\bar{I} = \bar{c}^2 \sum_i \frac{I_i}{c_i^2}$$

con

$$\bar{c} = \sqrt{\frac{1}{\sum_i \frac{1}{c_i^2}}}.$$

Es decir, se hace una media ponderada de cada estimación dando más importancia a las I_i tales que su c_i^2 es menor. El error esperado es (con una confianza del 99 %):

$$|\bar{I} - I_f| \leq 2,58 \cdot \bar{c}$$

Cabe destacar que, generar una muestra siguiendo la función de densidad (4.3) es sencillo: de manera independiente se genera una muestra x en la primera componente siguiendo q_x y una y en la segunda siguiendo q_y , la muestra siguiendo q_{xy} es (x, y) .

La generalización a d variables es obvia si consideramos la función de densidad:

$$q(x_1, \dots, x_d) = q_1(x_1) \dots q_d(x_d)$$

Aplicando el mismo razonamiento que con dos variables, VEGAS d -dimensional se sigue fácilmente.

4.2. MISER

Este algoritmo está basado en el muestreo estratificado; concretamente, su comportamiento se puede describir como un muestreo estratificado recursivo. Antes de dar una primera versión del pseudocódigo del algoritmo vamos a ilustrar un paso del mismo con un ejemplo sencillo. Pongamos que deseamos calcular la integral:

$$\int_{[a,b] \times [c,d]} f(x) dx.$$

Supongamos que estamos en una llamada al procedimiento MISER con un número de puntos p y una región r . Se realizaría lo que sigue:

- Si $p < umbral$ entonces se aplica *Media en el muestreo*. Esto es, se generan uniformemente p muestras y se toma

$$\bar{I}_f = V_r \frac{1}{p} \sum_{i=1}^p f(x_i)$$

$$\bar{\sigma}^2 = \frac{1}{p} \sum_{i=1}^p (V_r f(x_i) - \bar{I}_f)^2$$

entonces se devuelve el par $(\bar{I}_f, \frac{\bar{\sigma}^2}{p})$

- Si $p \geq umbral$, sea $m = 0,1 \cdot puntos$ y L el conjunto de posibles bisecciones, es decir si $r = [a, b] \times [c, d]$ entonces

$$L = \left\{ \left\{ \left[a, \frac{a+b}{2} \right] \times [c, d], \left[\frac{a+b}{2}, b \right] \times [c, d] \right\}, \left\{ [a, b] \times \left[c, \frac{c+d}{2} \right], [a, b] \times \left[\frac{c+d}{2}, d \right] \right\} \right\}$$

- Muestreemos r con m muestras obteniendo el conjunto $D = \{x_i\}_{i=1}^m$. Vamos a utilizar este conjunto para escoger la bisección que emplearemos en el siguiente paso recursivo. Para $L[1] = \left\{ [a, \frac{a+b}{2}] \times [c, d], [\frac{a+b}{2}, b] \times [c, d] \right\}$ se calcula:

$$A_{1,L[1]} = \frac{1}{|D_{1,L[1]}|} \sum_{x \in D_{1,L[1]}} f(x)$$

$$\sigma_{1,L[1]}^2 = \frac{1}{|D_{1,L[1]}|} \sum_{x \in D_{1,L[1]}} (f(x) - A_{1,L[1]})^2$$

con $D_{1,L[1]} = \{x \in D : x \in [a, \frac{a+b}{2}] \times [c, d]\}$. El cálculo de $\sigma_{2,L[1]}^2$, $\sigma_{1,L[2]}^2$, $\sigma_{2,L[2]}^2$ es análogo. Se escoge aquel $L[i]$ que minimice $\sigma_{1,L[i]} + \sigma_{2,L[i]}$.

- Finalmente se toman (n_1, n_2) tales que $n_1 + n_2 = 0,9 \cdot puntos$ y $\frac{n_1}{n_2} \approx \frac{\sigma_{1,L[i]}}{\sigma_{2,L[i]}}$ y se llama de nuevo al procedimiento MISER en cada una de las regiones determinadas por $L[i] = r_{i,1}, r_{i,2}$ con n_1 y n_2 .

Puede ocurrir que exista un i y j tal que $|D_{i,L[j]}| = 0$ o $\sigma_{i,L[j]}^2 = 0$. Entonces se escogería una bisección al azar y los puntos restantes $(0,9 \cdot puntos)$ se repartirían equitativamente.

El funcionamiento del algoritmo viene dado por el siguiente pseudocódigo:

Algoritmo 1 MISER**Data:** región, puntos**Result:** integral**if** *puntos* < *umbral* **then**| devolver (**Estimación Media en el muestreo**, **Estimación Varianza**/puntos) de región**else**

| numero_muestras := 0,1 · puntos

| divisiones := conjunto de bisecciones a lo largo de cada eje

| se muestrea la región con un número de muestras dado por numero_muestras

| se estima la desviación típica de todo par asociado a cada división

(región1, región2) := bisección que minimiza $(\sigma_1 + \sigma_2)$ (n_1, n_2) tales que $n_1 + n_2 = 0,9 \cdot \text{puntos}$ y $\frac{n_1}{n_2} \approx \frac{\sigma_1}{\sigma_2}$ (integralRegión1, varianzaRegión1) := MISER(región1, n_1)(integralRegión2, varianzaRegión2) := MISER(región2, n_2)

| devolver (integralRegión1 + integralRegión2, varianzaRegión1 + varianzaRegión2)

end

Sea la región rectangular R y supongamos que la hemos dividido en dos regiones R_1 y R_2 . Quemos aplicar el MI *Media en el muestreo* a cada región; luego consideremos Y_1 e Y_2 con $\bar{Y}_{n_1}$ e $\bar{Y}_{n_2}$ con $n_1 + n_2 = n$. Definimos $\bar{Y}_{st} = \bar{Y}_{n_1} + \bar{Y}_{n_2}$, por tanto se tiene que $\bar{Y}_{st}$ es el **estimador de muestreo estratificado** asociado a un $\{\omega_1, \omega_2\}$ donde $\omega_i(x) = \frac{1}{V_{R_i}}$ si $x \in V_{R_i}$.

En virtud del Teorema 3.13 la varianza $Var[\bar{Y}_{st}]$ se minimiza si tomamos:

$$n_1 = n \frac{\sigma_1}{\sigma_1 + \sigma_2}$$

$$n_2 = n \frac{\sigma_2}{\sigma_1 + \sigma_2}$$

y la varianza de $\bar{Y}_{st}$ vale:

$$\frac{(\sigma_1 + \sigma_2)^2}{n}, \quad (4.5)$$

siendo $\sigma_i = \sqrt{Var[Y_i]}$. En consecuencia si dividimos n_1 entre n_2 tenemos que:

$$\frac{n_1}{n_2} = \frac{\sigma_1}{\sigma_2} \quad (4.6)$$

La ecuación (4.5) justifica la elección de la bisección y la ecuación (4.6) justifica la elección de los puntos para continuar con la recursión. Lógicamente, el sistema formado por $n_1 + n_2 = 0,9 \cdot n$ y (4.6) no tiene soluciones necesariamente enteras, en consecuencia se toman los enteros cercanos a las soluciones reales. Las varianzas σ_1 y σ_2 se estiman utilizando el 10% de los puntos que se le ha pasado a la rutina como entrada, que se muestrean uniformemente sobre toda la región de entrada.

Ahora bien, el razonamiento anterior no es del todo correcto aplicarlo en un paso de MISER, pues estamos suponiendo que $\bar{Y}_{st}$ es un estimador de muestreo estratificado asociado a un par de unifomes $\{\omega_1, \omega_2\}$ y a un par de $\{n_1, n_2\}$ y, como hemos visto, MISER es un procedimiento recursivo y esto no sucede salvo que estemos justo un paso antes del caso base. Para intentar solucionar este problema, en [10] se propone tomar y minimizar:

$$Var[\bar{Y}'_{st}] = \frac{Var[Y_1]}{n_1^\alpha} + \frac{Var[Y_2]}{n_2^\alpha},$$

que se minimiza tomando:

$$n_i = n \frac{Var[Y_i]^{\frac{1}{1+\alpha}}}{Var[Y_1]^{\frac{1}{1+\alpha}} + Var[Y_2]^{\frac{1}{1+\alpha}}}.$$

De modo que la varianza $Var[\bar{Y}'_{st}]$ vale:

$$\frac{\left(Var[Y_1]^{\frac{1}{1+\alpha}} + Var[Y_2]^{\frac{1}{1+\alpha}}\right)^{1+\alpha}}{n^\alpha}$$

Aquí $\alpha \geq 1$ es una constante (se suele tomar $\alpha = 2$). Por consiguiente, el algoritmo MISER quedaría como sigue:

Algoritmo 2 MISER

Data: región, puntos

Result: integral

if *puntos* < *umbral* **then**

 | devolver (**Estimación Media en el muestreo, Estimación Varianza**/puntos) de región

else

 numero_muestras := 0,1 · puntos

 divisiones := conjunto de bisecciones a lo largo de cada eje

 se muestrea la región con un número de muestras dado por numero_muestras

 se estima la varianza de todo par asociado a cada división

 (región1, región2) := bisección que minimiza $(Var_1^{\frac{1}{1+\alpha}} + Var_2^{\frac{1}{1+\alpha}})^{1+\alpha}$

(n_1, n_2) tales que $n_1 + n_2 = 0,9 \cdot puntos$ y $\frac{n_1}{n_2} \approx \frac{Var_1^{\frac{1}{1+\alpha}}}{Var_2^{\frac{1}{1+\alpha}}}$

 (integralRegión1, varianzaRegión1) := MISER(región1, n_1)

 (integralRegión2, varianzaRegión2) := MISER(región2, n_2)

 devolver (integralRegión1 + integralRegión2, varianzaRegión1 + varianzaRegión2)

end

En la práctica, MISER no sólo devuelve la integral, también devuelve una estimación $\hat{\sigma}_{st}^2$ de la varianza de $\bar{Y}'_{st}$. Por consiguiente, el error esperado es (con una confianza del 99 %):

$$|\hat{I} - I_f| \leq 2,58 \cdot \hat{\sigma}_{st}.$$

4.3. Árbol de regresión

Según el Teorema 3.6, la función de distribución óptima viene dada por:

$$q(x) = \frac{|f(x)|}{\int_R |f(z)| dz}.$$

Esto es $Var_q[Y] \leq Var_\omega[Y]$, para toda función de densidad ω tal que $supp(f) \subset supp(\omega)$ con $\int_R \frac{f^2(x)}{\omega(x)} dx < \infty$. Supongamos que podemos aproximar f por g , siendo g una función tal que la distribución

$$q_g(x) = \frac{|g(x)|}{\int_R |g(z)| dz} \tag{4.7}$$

sea sencilla en el siguiente sentido: podemos generar números aleatorios que sigan la distribución descrita por q_g sin dificultad. Aquí es donde entran en juego los árboles de regresión.

Grosso modo, un árbol de regresión nos da una función que aproxima a f de la forma

$$g(x) = \sum_{i=1}^k a_i \chi_{R_i}(x),$$

donde R_i son rectángulos que particionan el dominio de f es decir, $\bigcup_{i=1}^k R_i = R$ y $R_i \cap R_j = \emptyset$ para todo $i \neq j$ (el procedimiento de obtención de la función g a partir de un conjunto de muestras $\{(x_t, f(x_t))\}_t$ se puede consultar en [17]). Así pues, podemos reescribir la expresión (4.7) como sigue:

$$q_g(x) = \frac{1}{\sum_{i=1}^k |a_i| V_{R_i}} \sum_{i=1}^k |a_i| \chi_{R_i}(x).$$

Generar números aleatorios siguiendo esa distribución es equivalente a seleccionar un rectángulo con probabilidad proporcional a $|a_i| V_{R_i}$ y luego tomar una muestra siguiendo una distribución $U(\text{int}(R_i))$.

Sin embargo, se nos presenta un problema ¿qué muestras usamos para entrenar el árbol de regresión? El Teorema 3.6 nos sugiere que tenemos que dar más atención a zonas del dominio tal que la función $|f(x)|$ tenga valores elevados. Así pues, se propone seguir los siguientes pasos:

1. Se generan M muestras siguiendo una distribución uniforme y se evalúan. Así se forma el conjunto de datos de entrenamiento: $D_{\text{train}} = \{(x_i, f(x_i))\}_{i=1}^M$.
2. Repetir hasta un número máximo de iteraciones:
 - a) Entrenar el árbol con el conjunto D_{train} y obtener la función g .
 - b) Usando la función de densidad q_g descrita en (4.7), generar M muestras y se evalúan obteniendo D' .
 - c) $D_{\text{train}} \leftarrow D_{\text{train}} \cup D'$.
3. A la salida del bucle anterior tenemos un árbol entrenado y, por consiguiente, una función de densidad q_g . Usando esa función de densidad se generan M' muestras y se evalúan obteniendo $\{(x_i, f(x_i))\}_{i=1}^{M'}$. Estas muestras se usan para estimar el valor de la integral y la desviación típica:

$$\hat{I} = \frac{1}{M'} \sum_{i=1}^{M'} \frac{f(x_i)}{q_g(x_i)}$$

$$\hat{\sigma} = \sqrt{\frac{1}{M'} \sum_{i=1}^{M'} \left(\frac{f(x_i)}{q_g(x_i)} - I \right)^2}$$

Aplicando la Proposición 2.6, podemos dar una cota aproximada del error (con una confianza del 99 %):

$$|\hat{I} - I_f| \leq \frac{2,58 \cdot \hat{\sigma}}{\sqrt{M'}}.$$

La idea de este algoritmo es que el conjunto de datos de entrenamiento lo formen, en su mayoría, ejemplares pertenecientes a las zonas de R donde la $|f(x)|$ sea grande; así, el árbol entrenado tendrá un refinamiento mayor en dichas zonas.

Observación 4.1:

Para construir un árbol de regresión utilizando las librerías de *scikit* es necesario, además de introducir las muestras, proporcionar una serie de hiperparámetros [18]. En particular, hay dos hiperparámetros que influyen mucho en la calidad de aproximación del árbol:

- *min_samples_leaf*: establece el mínimo número de muestras va a tener una hoja.
- *max_depth*: establece la altura máxima del árbol.

La función que implementa la integración con árboles presente en el Anexo 7 recibe estos dos hiperparámetros como argumentos permitiendo al usuario cambiarlos según sus necesidades. Nótese que no se puede saber los hiperparámetros óptimos, pues depende de la función a integrar, el número de dimensiones, el número de muestras, etc. Así pues, se deja a responsabilidad del usuario la elección de esos dos parámetros. En los experimentos en los que se usa este método de integración, he utilizado los siguientes valores:

- *min_samples_leaf* = 1
- *max_depth* = *None*, es decir, sin límite de altura.

Cabe destacar que estos parámetros no son los mejores (de hecho, pueden favorecer el *overfitting* y comprometer la calidad de q_g pues la aproximación g podría no aproximar bien f).

Ejemplo 4.2:

Vamos a emplear este método para calcular la integral:

$$\int_0^2 \int_0^2 \int_0^2 \int_0^2 x^2 + y^2 + z^2 + t^2 dx dy dz dt$$

Vamos a utilizar los siguientes parámetros:

- Número de iteraciones: 5
- $M = 5000$
- $M' = 75000$

En consecuencia, estaremos empleando un total de 100000 evaluaciones. Fijamos $z_\gamma = 2,58$ obtenemos en una ejecución lo siguiente:

- $\bar{I} = 85,383$ e $I = \frac{256}{3}$
- $|\bar{I} - I_f| = 0,05$
- $\hat{\sigma}^2 = 524,76$ y $\frac{2,58 \cdot \hat{\sigma}}{\sqrt{75000}} = 0,204$

En cambio, si aplicamos *Media en el muestreo* con $n = 100000$ obtenemos lo siguiente:

- $S_{exp} = 85,210$
- $|M_{exp} - I| = 0,12$

- $s_{S,exp}^2 = 1448,204$
- Tomamos la expresión de la Proposición 2.6: $z_\gamma \cdot s_S \cdot n^{\frac{-1}{2}} \approx 0,310$

Como se puede observar, la varianza estimada utilizando el método de los árboles es más pequeña que la estimada en el método de *Media en el muestreo*. Sin embargo, el error es muy parecido (pero ligeramente menor en el primer método) ya que la estimación de la integral se realiza con solo 75000 muestras en el primer caso y, en el segundo, se realiza con 100000 muestras.

Ejemplo 4.3:

Para ver que, efectivamente, este método se comporta como se espera (es decir, las muestras generadas a partir de q_g deben estar en zonas donde el valor absoluto de la función sea grande). Calculamos entonces la siguiente integral:

$$\int_{-2}^2 \int_{-2}^2 x^2 y^2 dx dy = \frac{256}{9} \approx 28,444$$

Tomamos los siguientes parámetros:

- Número de iteraciones: 5
- $M = 150$
- $M' = 2250$

En total usamos 3000 muestras. La integral estimada es $\hat{I} = 28,441$ y las muestras obtenidas se pueden visualizar en el siguiente gráfico:

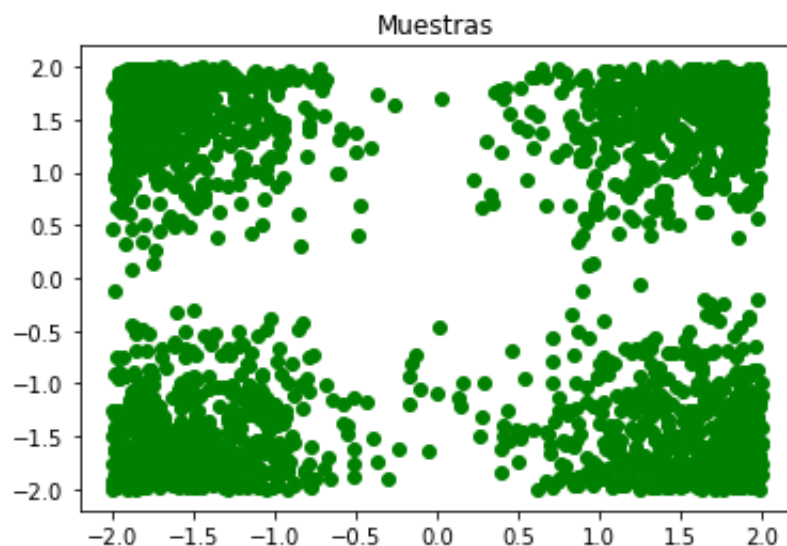


Figura 4.1: Muestras obtenidas a partir de la p_g obtenida en la última iteración.

Efectivamente, la función $f(x, y) = x^2y^2$ tiene alcanza sus valores mas grandes conforme nos vamos acercando a las esquinas del rectángulo $[-2, 2] \times [-2, 2]$.

Observación 4.4:

En este método se busca reducir la estimación del error disminuyendo la varianza; es decir:

$$\frac{\hat{\sigma}}{\sqrt{M'}} \leq \frac{s_S}{\sqrt{n}}$$

donde n es el número de muestras disponibles, M' el número de muestras que empleamos una vez entrenado el árbol ($M' < n$) y s_S es una estimación de la varianza una vez aplicado *Media en el muestreo*. De esta forma, para obtener p_g sacrifica un número de muestras. Se nos pueden presentar las siguientes situaciones:

- Se produce una reducción de varianza notable que implica una reducción del error con respecto a *Media en el muestreo*. Esto sería lo deseable.
- Se produce una reducción de varianza pero el error no es menor que en el caso de *Media en el muestreo*. Esto suele ocurrir cuando la reducción de varianza es pequeña.
- No se produce una reducción de varianza. Esto suele pasar cuando la función q_g no es buena.

4.4. *StackMonteCarlo*

Este algoritmo realmente es un método de post-procesamiento es decir, se supone que se dispone de un conjunto de muestras generadas siguiendo una determinada distribución de probabilidad ($X \sim \omega$). Esta técnica está basada en el aprendizaje supervisado y en la validación cruzada.

Más formamente, sea f una función integrable sobre un rectángulo d -dimensional R y, como siempre, estamos interesados en calcular la integral de f sobre ese rectángulo. Supongamos que tenemos un conjunto de muestras de tamaño n (llamemos D al conjunto de muestras) siguiendo una determinada distribución de probabilidad $X \sim \omega$. Supongamos g una función que aproxima f obtenida de alguna manera. Entonces, como desarrollamos en la técnica de reducción de varianza *Variables de control* se verifica lo siguiente:

$$\int_R f(x) dx = \alpha \int_R g(x) dx + E \left[\frac{f(X) - \alpha g(X)}{\omega(X)} \right].$$

La cuestión es cómo obtener g y α a partir de D . En primer lugar hay que fijar qué tipo de función será g (polinomio, suma de funciones simples, series de Fourier, ...) y determinar el algoritmo que se usará para obtener g . Supongamos que ya hemos resuelto ese trámite. Una solución *naïve* sería utilizar todo el conjunto D como entrada al algoritmo y obtener g ; pero esto generaría el temido *overfitting* o sobreentrenamiento, esto es, g va a aproximar muy bien los datos con los que se ha entrenado pero no funcionará bien con datos que no conozca. Además, el sobreentrenamiento puede llevarnos a una inexacta estimación del parámetro α . Para evitar este indeseable efecto, se propone usar validación cruzada cruzada de k -pliegues.

La validación cruzada cruzada de k -pliegues consiste en particionar el conjunto D (supongamos que $|D| = N$) en k conjuntos de *test* $D^{i,t}$ con $i = 1 \dots k$ que son disjuntos dos a dos y contienen el mismo número de muestras (salvo si N/k no es entero). El conjunto de entrenamiento para cada uno de los pliegues viene dado por $D^{i,e} = D - D^{i,t}$. Llamemos m_i y M_i al número de muestras de $D^{i,t}$ y $D^{i,e}$ respectivamente. Para cada pliegue $i = 1 \dots k$ se utilizan los elementos de $D^{i,e}$ como entrada al algoritmo de aprendizaje y se obtiene g_i una aproximación de f . Cada g_i se evalúa en $D^{i,t}$ y siguiendo algún criterio, se escoge un modelo final g (de entre los g_i).

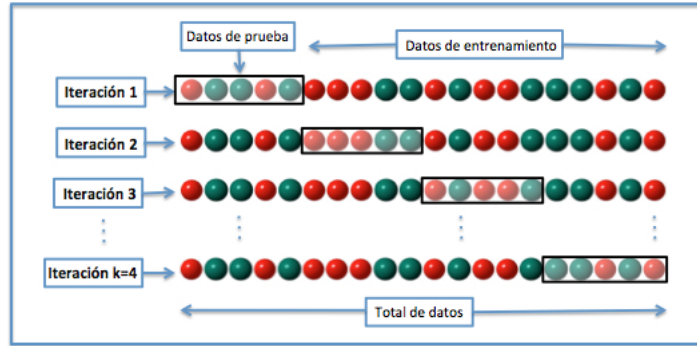


Figura 4.2: Validación cruzada de k -pliegues.

En *StackMonteCarlo* se propone llevar a cabo los siguientes pasos:

1. Fijamos el tipo de función g aproximadora de f (cuya integral sea fácil de calcular) y el algoritmo que nos genere dicha función.
2. Se aplica una validación cruzada cruzada de k -pliegues generando así $\{g_i\}_{i=1}^k$ funciones aproximadoras de f .
3. Se generarán un total de k estimaciones de la integral (usando las muestras del conjunto de test):

$$\hat{I}_f(i) = \alpha I_{g_i} + \frac{1}{m_i} \sum_{j=1}^{m_i} \frac{f(D^{i,t}(j)) - \alpha g_i(D^{i,t}(j))}{\omega(D^{i,t}(j))}.$$

Por la Proposición 2.6, el error previsto viene dado por:

$$|\hat{I}_f(i) - I_f| \leq e(i) = 2,58 \frac{\hat{\sigma}(i)}{\sqrt{m_i}}$$

$$\hat{\sigma}(i) = \sqrt{\frac{1}{m_i} \sum_{j=1}^{m_i} \left(\frac{f(D^{i,t}(j)) - \alpha g_i(D^{i,t}(j))}{\omega(D^{i,t}(j))} - \hat{I}_f(i) \right)^2}$$

4. Finalmente, se tomará como estimación final de la integral la media de las estimaciones de cada uno de los pliegues:

$$\hat{I}_f = \frac{1}{k} \sum_{i=1}^k \hat{I}_f(i),$$

con un error previsto de:

$$e = \max_{i=1 \dots k} e(i)$$

La estimación de los parámetros necesarios para calcular α se hará de la siguiente manera:

$$\begin{aligned}\mu_f &= \frac{1}{N} \sum_{j=1}^N \frac{f(D(j))}{\omega(D(j))} \\ \mu_g &= \frac{1}{N} \sum_{i=1}^k \sum_{j=1}^{m_i} \frac{g_i(D^{i,t}(j))}{\omega(D^{i,t}(j))} \\ \sigma_g^2 &= \frac{1}{N} \sum_{i=1}^k \sum_{j=1}^{m_i} \left(\frac{g_i(D^{i,t}(j))}{\omega(D^{i,t}(j))} - \mu_g \right)^2 \\ \sigma_{f,g} &= \frac{1}{N} \sum_{i=1}^k \sum_{j=1}^{m_i} \left(\frac{g_i(D^{i,t}(j))}{\omega(D^{i,t}(j))} - \mu_g \right) \left(\frac{f(D^{i,t}(j))}{\omega(D^{i,t}(j))} - \mu_f \right).\end{aligned}$$

Por consiguiente, siguiendo la fórmula presente en el Teorema 3.17, el parámetro α se toma como sigue:

$$\alpha = \frac{\sigma_{f,g}}{\sigma_g^2}$$

Nótese que las estimaciones se hacen trabajando sobre el conjunto de muestras de test. Esto es así ya que si las mediciones las hiciéramos sobre el conjunto de datos de entrenamiento el parámetro α sería grande, pues las funciones g_i arproximan bien f en esos puntos.

Podría ocurrir que alguna función g_i cuya integral diste mucho de I_f pero α tenga un valor alto debido a las buenas predicciones en el conjunto de test. En *StackMonteCarlo* se propone llevar una medida L_i para uno de los k pliegues dada por:

$$L_i = \frac{|I_{g_i} - I_{mc}|}{\bar{\sigma}},$$

donde I_{mc} es la estimación por el método de *Media en el muestreo* generalizado aplicado en el conjunto D (en otras palabras, $I_{mc} = \mu_f$) y $\bar{\sigma}$ viene dada por:

$$\begin{aligned}\bar{\sigma} &= \frac{\sigma_f}{\sqrt{N}} \\ \sigma_f &= \sqrt{\frac{1}{N} \sum_{j=1}^N \left(\frac{f(D(j))}{\omega(D(j))} - \mu_f \right)^2}.\end{aligned}$$

Es decir, σ_f^2 es la varianza muestral del método de *Media en el muestreo* generalizado aplicado en el conjunto D . Cuanto mayor sea L_i menos probable es que $I_{g_i} = I_f$ y, por consiguiente, la función g_i no aproxima bien f . En [12] se sugiere que si $\max_i \{L_i\} > 5$ entonces se rechace $\hat{I}_f$ y se tome como estimación de la integral I_{mc} .

Ejemplo 4.5:

Vamos a calcular la integral:

$$\int_0^\pi \int_0^\pi \int_0^\pi \int_0^\pi x^5 \sin(x) \sin(y) \sin(z) \sin(t) dx dy dz dt = 8 \left(\pi^5 - 20 (\pi^3 - 6\pi) \right)$$

Fijaremos los siguientes parámetros:

- Usaremos árboles de regresión para obtener las funciones g_i .

- Usaremos un conjunto D cuyo cardinal es de 50000 elementos. Las muestras presentes en el susodicho conjunto han sido generadas siguiendo una distribución uniforme ($\omega(x) = \frac{1}{\pi^4}$).
- Usaremos 5 pliegues, por consiguiente se tiene que $N = 50000$, $m_i = 10000$ y $M_i = 40000$ para $i = 1 \dots 5$.

Realizamos una ejecución del *StackMonteCarlo* y obtenemos los siguientes resultados:

- $\hat{I}_f = 504,304$ con un error estimado de $e = 3,717$
- $\alpha = 0,986$
- Se comete un error real de $|\hat{I}_f - I_f| = 1,222$
- $L = [0,545, 0,538, 0,471, 0,456, 0,520]$, se verifica que, para todo $i = 1 \dots 5$, $L_i < 5$. Así pues se acepta como válida la integral estimada por *StackMonteCarlo*.

Ejemplo 4.6:

Vamos a utilizar la integral del ejemplo anterior para comparar *Media en el muestreo* con *StackMonteCarlo*. Para ello, vamos a emplear diferentes conjuntos D que se irán modificando en cada iteración. En la primera iteración tendremos un total 1000 muestras, en la siguiente iteración 1500, así hasta llegar a un total de 15000 muestras. Obtenemos dos gráficas, una para el error real relativo ($\frac{|\hat{I}_f - I_f|}{I_f}$) y otra para el error estimado que, en el caso de *Media en el muestreo* es $2,58 \cdot s_g \cdot n^{-\frac{1}{2}}$ y, en el caso de *StackMonteCarlo* es $e = \max_{i=1 \dots k} e(i)$. Usaremos en cada iteración un total de 5 pliegues y, para obtener las g_i usaremos árboles de regresión. Así pues, obtenemos lo siguiente:

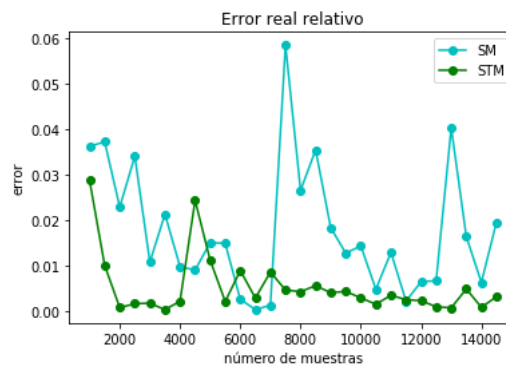


Figura 4.3: Error relativo real en cada iteración.

Como se puede observar, no siempre obtenemos un menor error real usando *StackMonteCarlo*. Este fenómeno es debido al componente aleatorio y no determinista de este tipo de métodos. Sin embargo, los resultados más interesantes están representados en la siguiente gráfica:

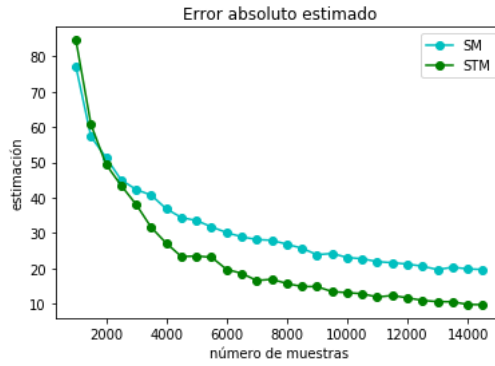


Figura 4.4: Error absoluto estimado en cada iteración.

La cota de error estimada por *StackMonteCarlo* es mejor que la cota estimada por *Sample-Mean*.

Antes de finalizar esta sección cabe decir que el método de *StackMonteCarlo* está pensado para funciones que tengan una varianza grande pues, como hemos visto, internamente aplica la técnica de reducción de varianza *Variables de control*. Una desventaja de este método es que depende mucho del modelo empleado para obtener las funciones g_i ya que, si las aproximaciones son muy malas, la estimación final del método también lo será. Sin embargo, tenemos la medida L_i para cada pliegue que nos ayuda a establecer la bondad de la estimación proporcionada por *StackMonteCarlo*.

Observación 4.7:

La rutina *StackMonteCarlo* utilizada en los ejemplos está implementada en *python* y puede encontrarse en el Anexo 7.

4.5. Comparación de métodos

El objetivo de esta última sección es comparar los métodos VEGAS, MISER, árboles y *Media en el muestreo* usando diferentes funciones definidas en el rectángulo d -dimensional $[0, 1]^d$.

Para cada uno de los cuatro métodos, se tendrán en cuenta las siguientes magnitudes:

- Número de muestras.
- Error real de la estimación.
- Estimación del error con una confianza del 99 %.

Así pues, haremos lo siguiente:

1. Tomaremos 4 funciones $\{f_i\}_{i=1}^4$ donde:

- f_1 es una función *sencilla*.
- f_2 es una función que no es cuadrado integrable en $[0, 1]^d$.
- f_3 es una función producto de exponenciales.
- f_4 es una suma de funciones trigonométricas.

2. Escogeremos un número de muestras inicial y lo variaremos anotando el error real de la estimación y la estimación del error.
3. Sacaremos conclusiones de las anotaciones tomadas.

Observación 4.8:

La rutina MISER utilizada está escrita en *python* y puede encontrarse en [15]. La rutina VEGAS puede descargarse en [14]. La rutina de árbol y *Media en el muestreo* puede consultarse en el Anexo 7.

Ejemplo 4.9:

Calcularemos la siguiente integral:

$$\int_{[0,1]^d} 2^d \prod_{i=1}^d x_i dx = 1.$$

Tomaremos $d = 4$. Los números de muestras que se van a utilizar son:

$$n \in \{5000, 10000, 15000, 20000, 25000, 30000, 35000, 40000\}$$

se llamarán a los siguientes métodos:

- Método de integración con árboles n muestras y 4 iteraciones.
- MISER con el número de muestras correspondiente.
- VEGAS de 10 iteraciones con $\frac{n}{10}$ muestras en cada iteración.
- *Media en el muestreo* con n muestras.

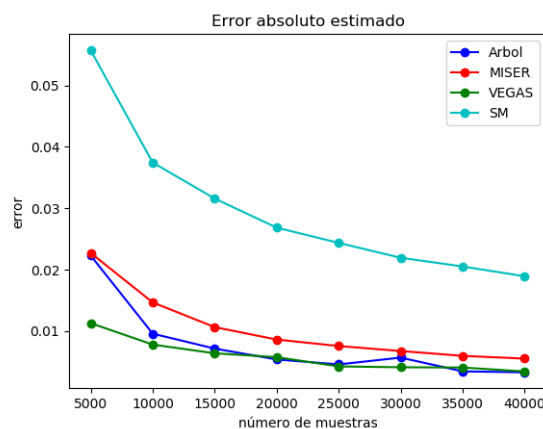


Figura 4.5: Error absoluto estimado.

Puede observarse en la gráfica anterior la menor estimación es la proporcionada por VEGAS. El método usando árboles tiene una estimación situada entre MISER y VEGAS. VEGAS,

MISER y Árbol tienen una estimación notablemente más pequeña que *Media en el muestreo* (SM).

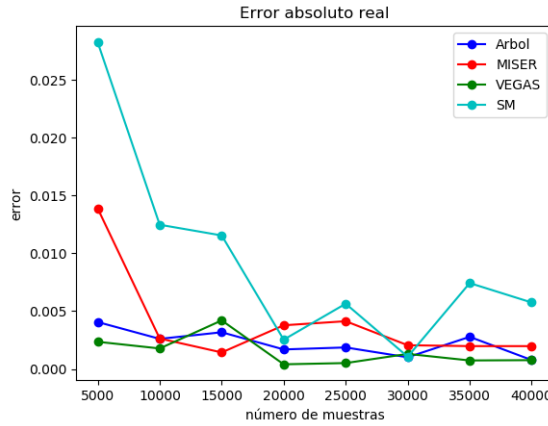


Figura 4.6: Error absoluto real.

Las cotas de error establecidas en la estimaciones se respetan y se observa la superioridad de VEGAS, MISER y Árbol con respecto a *Media en el muestreo*.

Ejemplo 4.10:

Calcularemos la siguiente integral:

$$\int_{[0,1]^d} \left(\frac{1}{2}\right)^d \prod_{i=1}^d \frac{1}{\sqrt{x_i}} dx = 1.$$

Tomaremos $d = 5$. Los números de muestras que se van a utilizar son:

$$n \in \{5000, 10000, 15000, 20000, 25000, 30000, 35000, 40000\}$$

se llamarán a los siguientes métodos:

- Método de integración con árboles n muestras y 4 iteraciones.
- MISER con el número de muestras correspondiente.
- VEGAS de 10 iteraciones con $\frac{n}{10}$ muestras en cada iteración.
- *Media en el muestreo* con n muestras.

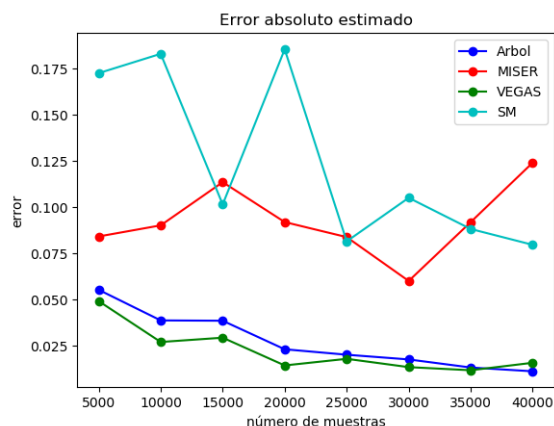


Figura 4.7: Error absoluto estimado.

En la mayoría de los casos, VEGAS proporciona la estimación del error más pequeña. En este ejemplo, Árbol establece una estimación más pequeña que MISER.

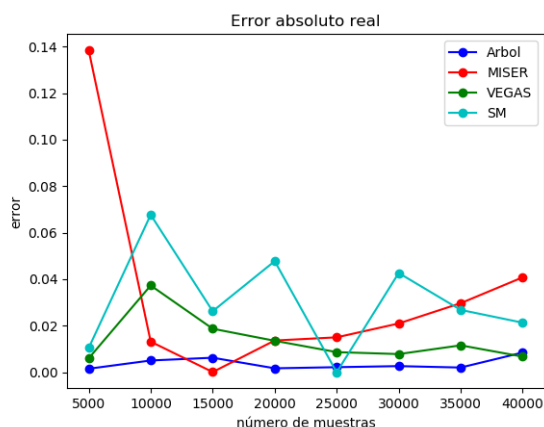


Figura 4.8: Error absoluto real.

Cabe destacar que la función que estamos integrando no es cuadrado integrable en $[0, 1]^d$ luego no se puede aplicar el Teorema Central del Límite para establecer una cota del error. Debido a esto, en algunas ejecuciones no se respeta la cota del error (es decir, el error absoluto real supera a la estimación) en el caso de VEGAS, *Media en el muestreo* y MISER. Sin embargo, en el caso de Árbol se respeta la estimación. Pese a este inconveniente, usando VEGAS y Árbol se obtienen buenos resultados en cuanto a error absoluto se refiere.

Ejemplo 4.11:

Calcularemos la integral:

$$\int_{[0,1]^d} \frac{1}{(e-1)^d} \exp \left\{ \sum_{i=1}^d x_i \right\} dx = 1.$$

Tomaremos $d = 8$. Los números de muestras que se van a utilizar son:

$$n \in \{5000, 10000, 15000, 20000, 25000, 30000, 35000, 40000, 45000\}$$

se llamarán a los siguientes métodos:

- Método de integración con árboles n muestras y 4 iteraciones.
- MISER con el número de muestras correspondiente.
- VEGAS de 10 iteraciones con $\frac{n}{10}$ muestras en cada iteración.
- *Media en el muestreo* con n muestras.

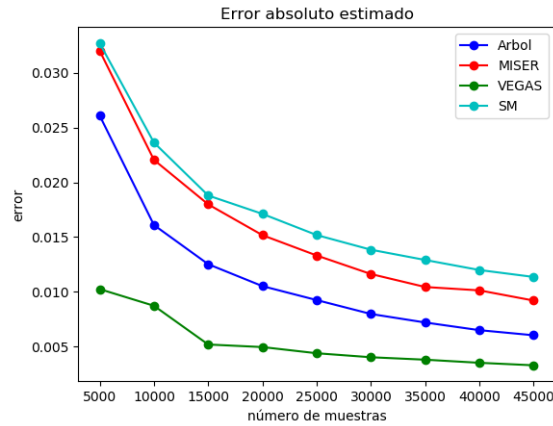


Figura 4.9: Error absoluto estimado.

Puede observarse en la gráfica anterior la menor estimación es la proporcionada por VEGAS. El método usando árboles proporciona una estimación menor que MISER, MISER y Árbol tienen una estimación más pequeña que *Media en el muestreo* (SM).

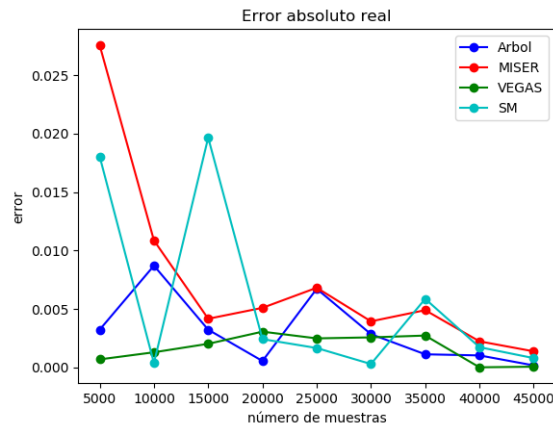


Figura 4.10: Error absoluto real.

Ejemplo 4.12:

Calcularemos la integral:

$$\int_{[0,2k\pi]^d} \sum_{i=1}^d \sin(x_i) dx = \int_{[0,1]^d} 2k\pi \sum_{i=1}^d \sin(2k\pi y_i) dy = 0.$$

Tomaremos $d = 6$ y $k = 1$. Los números de muestras que se van a utilizar son:

$$n \in \{5000, 10000, 15000, 20000, 25000, 30000, 35000, 40000, 45000\}$$

se llamarán a los siguientes métodos:

- Método de integración con árboles n muestras y 4 iteraciones.
- MISER con el número de muestras correspondiente.
- VEGAS de 10 iteraciones con $\frac{n}{10}$ muestras en cada iteración.
- *Media en el muestreo* con n muestras.

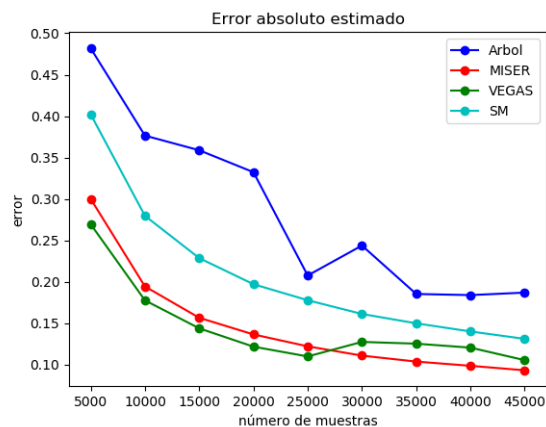


Figura 4.11: Error absoluto estimado.

En cuanto a estimación del error, MISER y VEGAS obtienen resultado similares. MISER, VEGAS y tienen una estimación menor que *Media en el muestreo*. Sin embargo, en este ejemplo, Árbol proporciona una estimación peor que *Media en el muestreo*, las posibles razones detrás de esto están expuestas en la Observación 4.4.

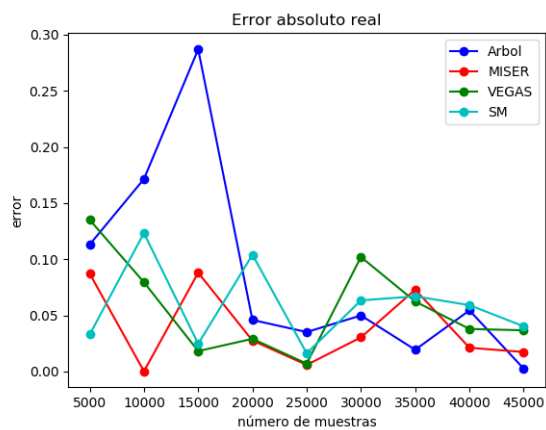


Figura 4.12: Error absoluto real.

El error real es muy inestable, especialmente, en el método de Árbol, esto es lógico puesto que la estimación del error es grande.

5 | Conclusiones

En este trabajo se ha estudiado el método de Monte Carlo para integración. En el Capítulo 1 introducimos el cálculo de volúmenes d -dimensionales, establecemos un método de estimación del volumen de un sólido y damos dos cotas del error: una teórica y una práctica.

En el Capítulo 2 nos centramos en el cálculo de integrales e introducimos, de manera formal, lo que se entiende por método de Monte Carlo de integración. Además estudiamos y comparamos dos métodos: *Acierto o fallo* y *Media en el muestreo*. Concluyendo, tanto teórica como experimentalmente, que este último era mejor.

Las cotas de error vistas en los dos capítulos anteriores se basan en la aplicación de Teorema Central del Límite; por lo cual, dichas cotas son pobres. Esto motiva la necesidad de introducir, en el Capítulo 3, las técnicas de reducción de varianza. En particular, nosotros vemos tres: *importancia en el muestreo* óptima, muestreo estratificado y *variables de control*.

En el Capítulo 4, se introducen algunas técnicas y/o algoritmos de integración que se emplean actualmente: VEGAS, MISER y *StackMonteCarlo*. Además, se intenta aplicar los árboles de regresión junto a *importancia en el muestreo* óptima para idear un método de integración competente. Finalmente, se comparan VEGAS, MISER, el método que emplea árboles de regresión y *Media en el muestreo* aplicándolos a cuatro funciones definidas en el rectángulo $[0, 1]^d$. Concluimos que el método más competente es VEGAS y que el método de los árboles depende mucho de la calidad de la aproximación y puede llegar a empeorar la cota del error proporcionada por *Media en el muestreo* (el método más *básico*).

6 | Anexo A : Teoría de la Probabilidad

Los contenidos de este Anexo están basados en el libro [13].

Definición 6.1:

Sea $(F_n)_n$ una sucesión de funciones de distribución. Se dice que F_n converge débilmente a F si, para todo punto de continuidad x de F se verifica:

$$\lim_n F_n(x) = F(x).$$

Esto implica que:

$$\lim_n F_n(x-) = F(x),$$

donde $G(x-) = \lim_{t \rightarrow x-} G(t)$.

Definición 6.2:

Fijado un espacio de probabilidad $(\Omega, \mathcal{S}, P)$. Dada una sucesión de variables aleatorias $(X_n)_{n \in \mathbb{N}}$ con $X_n : \Omega \rightarrow \mathbb{R}$ para todo $n \in \mathbb{N}$, entonces:

- Se dice que converge punto a punto a X si

$$\lim_n X_n(\omega) = X(\omega) \text{ para todo } \omega \in \Omega.$$

- Se dice que converge casi seguramente a X si

$$P\left(\lim_n X_n = X\right) = 1,$$

se representa de la siguiente forma: $X_n \rightarrow_{c.s.} X$.

- Se dice que converge en probabilidad a X si

$$\lim_n P(|X_n - X| < \varepsilon) = 1 \text{ para todo } \varepsilon > 0,$$

se representa de la siguiente forma: $X_n \rightarrow_P X$.

- Se dice que converge en distribución a X si

$$F_n \rightarrow F \text{ débilmente,}$$

donde F_n es la función de distribución de X_n y F es la de X . Se representa de la siguiente forma: $X_n \rightarrow_D X$.

Proposición 6.3:

Se tiene la siguiente cadena de implicaciones:

$$X_n \text{ converge punto a punto a } X \implies X_n \xrightarrow{c.s.} X \implies X_n \xrightarrow{P} X \implies X_n \xrightarrow{D} X$$

Teorema 6.4:

(Teorema de Lindeberg-Levy o Teorema Central del Límite) Sea $(X_n)_n$ una sucesión de variables aleatorias independientes idénticamente distribuidas de media μ y varianza $0 < \sigma^2 < \infty$. Llamemos $S_n = \sum_{i=1}^n X_i$. Entonces la variable aleatoria

$$\frac{S_n - n\mu}{\sqrt{n}\sigma}$$

converge en distribución a una variable aleatoria $N(0, 1)$.

Teorema 6.5:

(Ley Fuerte de los Grandes Números) Sea $(X_n)_n$ una sucesión de variables aleatorias independientes idénticamente distribuidas de media μ que cumplen $E[|X_i|] < \infty$. Llamemos $S_n = \sum_{i=1}^n X_i$. Entonces se verifica que

$$\frac{S_n}{n} \xrightarrow{c.s.} \mu.$$

Proposición 6.6:

(Teorema de Slutsky) Sean $(X_n)_n$ y $(Y_n)_n$ dos sucesiones de variables aleatorias tales que $X_n \xrightarrow{D} X$ y $Y_n \xrightarrow{P} c$ con c una constante. Entonces se verifica que:

- $X_n + Y_n \xrightarrow{D} X + c$.
- $X_n Y_n \xrightarrow{D} cX$.
- $X_n/Y_n \xrightarrow{D} X/c$ suponiendo que c es distinto de 0.

Proposición 6.7:

(Ley de Eve o Ley de la varianza total) Sean X e Y dos variables aleatorias. Supongamos $Var[Y] < \infty$. Entonces:

$$Var[Y] = E[Var[Y|X]] + Var[E[Y|X]].$$

Un caso especial, si $A_1, \dots, A_n$ es una partición mutuamente exclusiva y exhaustiva de sucesos del espacio muestral ($A_i \cap A_j = \emptyset$ y $\bigcup_{i=1}^n A_i = \Omega$), entonces:

$$\begin{aligned} \text{Var}[Y] &= \sum_{i=1}^n \text{Var}[Y|A_i]P(A_i) + \sum_{i=1}^n E[Y|A_i]^2(1 - P(A_i))P(A_i) \\ &\quad - 2 \sum_{i=2}^n \sum_{j=1}^{i-1} E[Y|A_i]P(A_i)E[Y|A_j]P(A_j). \end{aligned}$$

En esta fórmula, la primera componente es la esperanza de la varianza condicional; los otros dos sumandos son la varianza de la esperanza condicional.

Definición 6.8:

Sea X una variable aleatoria, se dice que el vector aleatorio de dimensión n , $(X_1, \dots, X_n)$, es una muestra aleatoria simple (m.a.s.) de tamaño n de X , si para cada variable X_j esta sigue la misma distribución que X y las variables X_j son independientes entre si.

Un buen estimador de la media es la media muestral:

Definición 6.9:

Sea X una variable aleatoria y $(X_1, \dots, X_n)$ una m.a.s. de X se define la media muestral como sigue:

$$\bar{X} = \frac{\sum_{i=1}^n X_i}{n}.$$

La media muestral es interesante ya que verifica las siguientes propiedades:

Proposición 6.10:

Sea X una variable aleatoria y $(X_1, \dots, X_n)$ una m.a.s. entonces:

- $E[\bar{X}] = E[X]$.
- $\text{Var}[\bar{X}] = \frac{\text{Var}[X]}{n}$.
- $\bar{X} \rightarrow_P E[X]$, suponiendo que $\text{Var}[X] < \infty$.

Demostración. Las dos primeras son directas:

- $E[\bar{X}] = n^{-1}E[\sum_{i=1}^n X_i] = n^{-1} \sum_{i=1}^n E[X_i] = n^{-1}nE[X] = E[X]$.
- $\text{Var}[\bar{X}] = n^{-2}\text{Var}[\sum_{i=1}^n X_i] = n^{-2} \sum_{i=1}^n \text{Var}[X_i] = n^{-2}n\text{Var}[X] = \frac{\text{Var}[X]}{n}$.

Para la última basta tener en cuenta la desigualdad de *Chebyshev*. Dada una variable aleatoria X se cumple que: $\forall \epsilon > 0$, $P(|X - E[X]| \geq \epsilon) \leq \frac{\text{Var}[X]}{\epsilon^2}$. Por tanto, fijado $\epsilon > 0$:

$$P(|\bar{X} - E[\bar{X}]| \geq \epsilon) \leq \frac{Var[\bar{X}]}{\epsilon^2} = \frac{Var[x]}{n\epsilon^2} \rightarrow 0 \text{ si } n \rightarrow \infty \text{ pues } Var[X] \text{ es una constante.}$$

□

Definición 6.11:

Sea X una variable aleatoria y $(X_1, \dots, X_n)$ una m.a.s. de X se define la varianza muestral como sigue:

$$s^2 = \frac{\sum_{i=1}^n (X_i - \bar{X})^2}{n} = \frac{\sum_{i=1}^n X_i^2}{n} - (\bar{X})^2$$

Este estimador es interesante porque cumple la siguiente propiedad:

Proposición 6.12:

Sea X una variable aleatoria y $(X_1, \dots, X_n)$ una m.a.s. de X . Entonces $s^2 \rightarrow_P Var(X)$.

Demostración. $s^2 = \frac{\sum_{i=1}^n X_i^2}{n} - (\bar{X})^2$, llamemos $Z_i = X_i^2$. La varianza muestral se puede reescribir como $s^2 = \bar{Z} - (\bar{X})^2$.

Por la ley fuerte de los grandes números, se tiene que:

$$\begin{aligned} \bar{Z} &\rightarrow_{c.s.} E[Z] \\ \bar{X}^2 &\rightarrow_{c.s.} E[X]^2 \end{aligned}$$

Por tanto, se tiene que $s^2 \rightarrow_{c.s.} E[Z] - E[X]^2$. Si desarrollamos lo anterior $E[Z] - E[X]^2 = E[X^2] - E[X]^2 = Var(X)$. Por consiguiente, $s^2 \rightarrow_{c.s.} Var[X]$ lo que implica que $s^2 \rightarrow_P Var[X]$. □

Observación 6.13:

En la proposición anterior se ha demostrado que $s^2 \rightarrow_{c.s.} Var[X]$ que es una condición más fuerte que la convergencia en probabilidad.

Observación 6.14:

Como $s^2 \rightarrow_{c.s.} Var(X)$ entonces $\sqrt{s^2} = s \rightarrow_{c.s.} \sqrt{Var[X]} = \sigma_X$ (la desviación típica). Esto implica que $s \rightarrow_P \sigma_X$

Vamos a utilizar el resultado anterior para demostrar la siguiente proposición:

Proposición 6.15:

Sea $X_1, \dots, X_n, \dots$ una sucesión de variables aleatorias, independientes e idénticamente distribuidas ($X_i \sim X$) de una distribución con media $E[X]$ y varianza $0 < Var[X] < \infty$. Entonces se verifica que:

$$\frac{\bar{X} - E[X]}{s/\sqrt{n}} \rightarrow_D Z \sim N(0, 1)$$

Demostración. Por el teorema central del límite se verifica que $\frac{\bar{X}-E[X]}{\sqrt{Var[\bar{X}]}} \rightarrow_D Z \sim N(0,1)$. Como se demostró previamente $Var[\bar{X}] = \frac{Var[X]}{n}$. $Var[X]$ es constante luego aplicando el teorema de *Slutsky* se verifica que:

$$\frac{\bar{X}-E[X]}{\sigma_X/\sqrt{n}} \cdot \sigma_X = \frac{\bar{X}-E[X]}{1/\sqrt{n}} \rightarrow_D Z' = Z \cdot \sigma_X \sim N(0, \sigma_X^2), \text{ ya que } Z \sim N(0,1).$$

Como $Var[X] > 0$ y $s \rightarrow_P \sigma_X$, aplicando de nuevo el teorema de *Slutsky*, podemos concluir que:

$$\frac{\bar{X}-E[X]}{1/\sqrt{n}} \cdot \frac{1}{s} = \frac{\bar{X}-E[X]}{s/\sqrt{n}} \rightarrow_D Z'' = Z' \cdot \frac{1}{\sigma_X} \sim N(0,1), \text{ ya que } Z' \sim N(0, \sigma_X^2)$$

□

7 | Anexo B : Código en python generado

7.1. Clase Árbol

```
1 from sklearn.tree import DecisionTreeRegressor
import numpy as np
3
5
6 ## Esta clase encapsula un árbol de regresión para integrar una función.
7 #
8 # Es utilizada por StackMonteCarlo y el método de árboles para integración.
9 class Arbol:
10
11     ## Constructor de la clase.
12     # @param self El puntero al objeto.
13     # @param depth Profundidad máxima del árbol.
14     # @param extremes Extremos de integración.
15     # @param minLeaf El mínimo número de muestras requeridas para se un nodo hoja.
16
17     def __init__( self , depth , extremes , minLeaf ) :
18         self.depth = depth
19         self.cart = DecisionTreeRegressor ( max_depth=depth , min_samples_leaf=minLeaf )
20         self.extrems = extremes
21         self.trained = False
22
23     def explorarArbolAux ( self , tabla , desigualdades , rectangulos , is_leaves ) :
24         if ( len ( tabla ) == 0 ) :
25             return
26         nodo = tabla [ 0 ]
27         if is_leaves [ nodo ] :
28             rectangulos . append ( desigualdades )
29             return
30         else :
31             des1 = desigualdades . copy ()
32             des2 = desigualdades . copy ()
33             des1 . append ( ( "<=" , self . threshold [ nodo ] , self . feature [ nodo ] ) )
34             des2 . append ( ( ">" , self . threshold [ nodo ] , self . feature [ nodo ] ) )
35             self . explorarArbolAux ( np . copy ( tabla [ 1 : ] ) , des1 , rectangulos , is_leaves )
36             derecho = int ( np . argwhere ( tabla == self . children_right [ nodo ] ) )
37             self . explorarArbolAux ( np . copy ( tabla [ derecho : ] ) , des2 , rectangulos ,
38                                     is_leaves )
39             return
40
41     def generarrectangulos ( self , limites ) :
42         rectangulo = np . copy ( self . extremes )
```



```

43     for (a,b,c) in limites:
44         if (a=="<="):
45             rectangulo[c,1] = min(b,rectangulo[c,1])
46         else:
47             rectangulo[c,0] = max(b,rectangulo[c,0])
48     return rectangulo
49
50 def explorarArbol (self):
51     rectangulos = []
52     n_nodes = self.nodes
53     children_left = self.children_left
54     children_right = self.children_right
55
56     node_depth = np.zeros(shape=n_nodes, dtype=np.int64)
57     is_leaves = np.zeros(shape=n_nodes, dtype=bool)
58     stack = [(0, -1)] # seed is the root node id and its parent depth
59     while len(stack) > 0:
60         node_id, parent_depth = stack.pop()
61         node_depth[node_id] = parent_depth + 1
62         # If we have a test node
63         if (children_left[node_id] != children_right[node_id]):
64             stack.append((children_left[node_id], parent_depth + 1))
65             stack.append((children_right[node_id], parent_depth + 1))
66         else:
67             is_leaves[node_id] = True
68     self.explorArbolAux(np.array(range(0,n_nodes)),list([]),rectangulos ,
69     is_leaves)
70     return [self.generarrectangulos(r) for r in rectangulos]
71
72 def valorrectangulo(self,r):
73     m = [((y-x)/2)+x for [x,y] in r]
74     return self.cart.predict(np.float32(np.array([m])),check_input = False)
75
76 def calcularAreas(self):
77     areas = []
78     for r in self.rectangulos:
79         aux = []
80         for e in r:
81             aux.append(e[1]-e[0])
82         areas.append(np.array(aux))
83     areas = np.array(areas)
84     areas = np.prod(areas,axis=1)
85     return areas
86
87 ## Método para entrenar el árbol de integración.
88 # @param self El puntero al objeto.
89 # @param X Conjunto de entrenamiento.
90 # @param y Valor esperado de los ejemplares de entrenamiento.
91 def train(self,X,y):
92     self.cart.fit(X,y)
93     self.trained = True
94     self.feature = self.cart.tree_.feature
95     self.children_left = self.cart.tree_.children_left
96     self.children_right = self.cart.tree_.children_right
97     self.threshold = self.cart.tree_.threshold
98     self.nodes = self.cart.tree_.node_count
99     self.rectangulos = self.explorArbol()
100     self.values = np.hstack([self.valorrectangulo(r) for r in self.rectangulos])
101     self.areas = self.calcularAreas()

```

```

101         self.ponderaciones = np.multiply(self.areas,np.abs(self.values))
102         total = np.sum(self.ponderaciones)
103         self.ponderaciones = self.ponderaciones / total
104         self.total = total
105         self.nleaves = len(self.ponderaciones)
106         self.integral = np.sum([x*y for x,y in zip(self.areas ,self.values)])
107
108
109     ## Método para generar muestras aleatorias siguiendo la distribución generada
110     ## por el árbol.
111     # @param self El puntero al objeto.
112     # @param nmuestras Número de muestras que se quiere generar.
113     # @return Muestras generadas.
114     def generarAleatorio(self ,nmuestras):
115         resultado = []
116         indexs = len(self.ponderaciones)
117         for i in range(nmuestras):
118             index = np.random.choice(np.arange(0,indexs) , p=self.ponderaciones)
119             r = self.rectangulos[index]
120             res = []
121             for [a,b] in r:
122                 res.append(np.random.uniform(a,b))
123             res = np.array(res)
124             resultado.append((res ,index))
125         return np.array(resultado)
126
127     def inRectangulo(self ,rectangulo ,x):
128         resultado = True
129         for d in range(len(x)):
130             resultado = resultado and (x[d]<=rectangulo[d,1]) and (x[d]>rectangulo[d
131             ,0])
132         return resultado
133
134     def pfx(self ,a,i):
135         valores = self.values
136         total = self.total
137         return abs(valores[i])*(1/total)
138
139     ## Función de distribución generada por el árbol.
140     # @param self El puntero al objeto.
141     # @param x valor real.
142     # @return pg(x).
143     def pfx2(self ,x):
144         valores = self.values
145         total = self.total
146         i = 0
147         for r in self.rectangulos:
148             if(self.inRectangulo(r,x)):
149                 return abs(valores[i])*(1/total)
150             ++i
151     ## Método para imprimir la información del árbol entrenado.
152     # @param self El puntero al objeto.
153     def printInformation(self):
154         print("Informacion actual del arbol:")
155         print("Nodos" ,self.nodes)
156         print("Rectangulos: " ,self.rectangulos)
157         print("Areas: " , self.areas)
158         print("Suma areas:" ,np.sum(self.areas))
159         print("Valores:" , self.values)

```

```

159     ## Método para devolver la puntuación del árbol entrenado.
161     # @param self El puntero al objeto.
163     # @param X Muestras de test.
165     # @param y Valor esperado de los ejemplares de test.
167     # @return Puntuación.
169     def score(self,X,y):
171         return self.cart.score(X,y)

173     ## Método que aplica la función generada por el árbol a un conjunto de valores.
175     # @param self El puntero al objeto.
177     # @param X Muestras a las que aplicar la función.
179     # @return y Valores que resultan de aplicar la función del árbol a la entrada.
181     def predict(self,X):
183         return self.cart.predict(X,check_input=False)

```

Clase que encapsula un árbol de regresión

7.2. Métodos de integración implementados

```

import numpy as np
import math
from ArbolClass import Arbol
from sklearn.model_selection import KFold
import MISER as mi
import vegas

## Esta clase recoge varios métodos de integración Monte Carlo.
#
#
class MetodosIntegracionMonteCarlo:

    ## Constructor de la clase.
    # @param self El puntero al objeto.
    def __init__(self):
        return

    ## Función que genera muestras d-dimensionales siguiendo una distribución
    uniforme.
    # @param self El puntero al objeto.
    # @param extremos Hipercubo donde se van a generara las muestras.
    # @param puntos Número de muestras a generar.
    # @param y Valor esperado de los ejemplares de entrenamiento.
    # @return Muestras.
    def generarMuestrasUniforme(self, extremos, puntos):
        return [[np.random.uniform(extremoA,extremoB) for [extremoA,extremoB] in
extremos] for i in range(puntos)]

    ## Función que devuelve el volumen de un hipercubo.
    # @param self El puntero al objeto.
    # @param extremos Hipercubo donde se van a generara las muestras.
    # @return Volumen.
    def volumen(self, extremos):
        medidas = [y-x for [x,y] in extremos]
        return np.prod(medidas)

    ## Función que calcula el volumen de un sólido d-dimensional.
    # @param self El puntero al objeto.

```

```

36 # @param pertenece Función que debe recibir como entrada un punto d-dimensional
   # e indica si pertenece o no al sólido.
38 # @param extremos Hipercubo en el cual se encuentra encerrado el sólido.
   # @param nmuestras Número de muestras a utilizar para calcular el volumen.
   # @return Par formado por el volumen y una estimación del error absoluto con
   una confianza del 99%.
40 def volumenMC(self, pertenece, extremos, nmuestras):
    muestras = np.array(self.generarMuestrasUniforme(extremos, nmuestras))
42    vol = self.volumen(extremos)
    muestrasAcertadas = np.array([x for x in muestras if pertenece(x)])
44    nAcertadas = len(muestrasAcertadas)
    nTotal = nmuestras
46    tita = float(nAcertadas)/float(nTotal)
    area = vol * tita
48    bexp = np.array([pertenece(x) for x in muestras])

50    return (area, 2.58*np.std(bexp)*vol/math.sqrt(nmuestras))

52 ## Función que calcula la integral de una función en un hipercubo usando el método
   de media en el muestreo.
   # @param self El puntero al objeto.
54 # @param func Función que se quiere integrar.
   # @param extr Hipercubo en el cual se quiere integrar.
56 # @param puntos Número de muestras a utilizar para integrar.
   # @return 3-upla formado por el volumen, una estimación de la varianza y una
   estimación del error absoluto con una confianza del 99%.
58 def integralSampleMean(self, func, extr, puntos):
    medidas = [y-x for [x,y] in extr]
60    volumen = np.prod(medidas)
    anadir = [[np.random.uniform(extremoA, extremoB) for [extremoA, extremoB] in
extr] for i in range(puntos)]
62    fpp = [volumen*func(x) for x in anadir]
    integralS = np.mean(fpp)
64    desvi = np.std(fpp)
    var = np.var(fpp)
66    estimacionError = 2.58*desvi/(math.sqrt(puntos))
    return (integralS, var, estimacionError)

68 ## Función que genera un conjunto de muestras. El dominio esta particionado, se
   escoge un hipercubo de la partición al azar siguiendo las ponderaciones de los
   mismos y luego se genera la muestra siguiendo una uniforme del hipercubo
   seleccionado.
   # @param self El puntero al objeto.
70 # @param ponderaciones Lista de valores entre cero y uno que indican el peso de
   cada hipercubo.
   # @param rectangulos Hipercubos de la partición.
72 # @param nmuestras Número de muestras a generar.
   # @return Conjunto de muestras generadas.
74 def generarAleatorio(self, ponderaciones, rectangulos, nmuestras):
    resultado = []
76    indexs = len(ponderaciones)
    for i in range(nmuestras):
78        index = np.random.choice(np.arange(0, indexs), p=ponderaciones)
        r = rectangulos[index]
80        res = []
        for [a,b] in r:
82            res.append(np.random.uniform(a, b))
        res = np.array(res)
84        resultado.append(res)
    return np.array(resultado)
86

```

```

88  ## Función que calcula la integral de una función en un hipercubo aplicando
muestreo estratificado a una función de distribución que es suma ponderada de
uniformes.
# @param self El puntero al objeto.
90 # @param f Función que se quiere integrar.
# @param rectangulos Hipercubos que definen la distribución.
92 # @param ponderaciones Lista de valores entre cero y uno que indican el peso de
cada hipercubo.
# @param n Número de muestras a utilizar para integrar.
94 # @return 4-upla formada por la integral, varianza del estimador, una estimación
del error absoluto con una confianza del 99% y número de muestras usadas.
def stratifiedSample(self, f, rectangulos, ponderaciones, n):
96     np.random.seed(1234)

98     i = 0

100     muestras = 0

102     integrales = []

104     varianzas = []

106     nis = []

108     for r in rectangulos:

110         ni = int(n*ponderaciones[i])

112         it, v, _ = self.integralSampleMean(f, r, ni)

114         integrales.append(it)

116         varianzas.append(v)

118         i = i + 1

120         muestras = muestras + ni

122         nis.append(ni)

124

126     v_st = np.sum(np.divide(varianzas, nis))
i_st = np.sum(integrales)

128     desvi = math.sqrt(v_st)
estimacionError = 2.58*desvi

130

132     return (i_st, v_st, estimacionError, muestras)

134

## Función auxiliar que dado un punto (x,y) establece si está en  $D^+$ ,  $D^-$  o en
ninguno.
# @param self El puntero al objeto.
# @param f Función que define los conjuntos.
138 # @param x Punto a clasificar.
# @return Devuelve 1 si (x,y) está en  $D^+$ , -1 si está en  $D^-$  o 0 en otro caso.
def criterio(self, f, x):
140     long = len(x)

```

```

142         xi = x[:long]
143         y = x[long-1]
144         fxmas = max(f(xi),0)
145         fxmenos = max(-f(xi),0)
146         if fxmas>0 and y<=fxmas:
147             return 1
148         if fxmenos>0 and y<=fxmenos:
149             return -1
150         return 0
151
152     ## Función que calcula la integral de una función en un hipercubo usando el mé
153     todo de Acierto o fallo.
154     # @param self El puntero al objeto.
155     # @param f Función que se quiere integrar.
156     # @param ext Hipercubo en el cual está encerrado el grafo de la función valor
157     absoluto. Las primeras componentes de la lista son los extremos del hipercubo y
158     la última componente es de la forma [0,c].
159     # @param nmuestras Número de muestras a utilizar para integrar.
160     # @return 3-upla formado por la integral, una estimación de la varianza y una
161     estimación del error absoluto con una confianza del 99%.
162     def hitandmiss(self,f,ext,nmuestras):
163         np.random.seed(1234)
164         extremos = ext
165         muestras = np.array([[np.random.uniform(extremoA,extremoB) for [extremoA,
166         extremoB] in extremos] for i in range(nmuestras)])
167         vol = self.volumen(extremos)
168         bexp = np.array([self.criterio(f,x) for x in muestras])
169         integral = np.mean(bexp)*vol
170         desvi = np.std(bexp) * vol
171         var = np.var(bexp) * (vol**2)
172         estimacionError = 2.58*desvi/(math.sqrt(nmuestras))
173         return (integral,var,estimacionError)
174
175     ## Función que calcula la integral de una función en un hipercubo usando árboles
176     de regresión.
177     # @param self El puntero al objeto.
178     # @param f Función que se quiere integrar.
179     # @param ext Hipercubo sobre el cual se quiere integrar la función.
180     # @param nmuestras Número de muestras a utilizar para integrar.
181     # @param niter Número de iteraciones que usará el método
182     # @param minLeaf=1 El mínimo número de muestras requeridas para se un nodo hoja
183     .
184     # @param depth=None Profundidad máxima del árbol.
185     # @return 3-upla formado por la integral, una estimación de la varianza y una
186     estimación del error absoluto con una confianza del 99%.
187     def integracionArbol(self,f,ext,nmuestras,niter,minLeaf=1,depth=None):
188
189         iterar = int(nmuestras / 4)
190
191         MPI = int(iterar / niter)
192         restantes = nmuestras - MPI * niter
193         miarbol = Arbol(depth=depth,extrems=ext,minLeaf=minLeaf)
194
195         muestras = [[np.random.uniform(extremoA,extremoB) for [extremoA,extremoB] in
196         ext] for i in range(MPI)]
197         muestrasT = np.array([np.array(x) for x in muestras])
198         fmuestras = np.array([f(x) for x in muestras])
199
200         for ite in range(niter):

```

```

194         miarbol.train(muestrasT, fmuestras)
195         anadir = miarbol.generarAleatorio(MPI)
196         yanadir = np.array([f(x) for (x,y) in anadir])
197         muestrasT = np.vstack([muestrasT, np.array([x for (x,y) in anadir])])
198         fmuestras = np.concatenate((fmuestras, yanadir))
199
200     muestras = miarbol.generarAleatorio(restantes)
201     fpp = [f(x)/miarbol.pfx(x,y) for (x,y) in muestras]
202     integral = np.mean(fpp)
203     std = np.std(fpp)
204     var = np.var(fpp)
205     estimacion_error = 2.58*std/math.sqrt(restantes)
206
207     return (integral, var, estimacion_error)
208
209
210     ## Función que calcula la integral de una función en un hipercubo usando VEGAS.
211     # @param self El puntero al objeto.
212     # @param f Función que se quiere integrar.
213     # @param extremos Hipercubo sobre el cual se quiere integrar la función.
214     # @param nmuestras Número de muestras a utilizar por iteración.
215     # @param niter Número de iteraciones que usará el método.
216     # @return Par formado por la integral y una estimación del error absoluto con
217     una confianza del 99%.
218     def VEGASInt(self, f, extremos, nmuestras, niter):
219         integ = vegas.Integrator(extremos)
220         result = integ(f, nitn=niter, neval=nmuestras)
221         return (result.mean, 2.58*result.sdev)
222
223     ## Función que calcula la integral de una función en un hipercubo usando MISER.
224     # @param self El puntero al objeto.
225     # @param f Función que se quiere integrar.
226     # @param extremos Hipercubo sobre el cual se quiere integrar la función.
227     # @param nmuestras Número de muestras a utilizar para integrar.
228     # @return Par formado por la integral y una estimación del error absoluto con
229     una confianza del 99%.
230     def MISERInt(self, f, extremos, nmuestras):
231         m = mi.MISER()
232
233         low = []
234         high = []
235
236         for [a,b] in extremos:
237             low.append(a)
238             high.append(b)
239
240         inte, var, points = m.MISER(f, low, high, nmuestras, 0)
241         return (inte, math.sqrt(var)*2.58)
242
243
244     ## Función que calcula la integral de una función en un hipercubo usando
245     StackMonteCarlo con una distribución uniforme y árboles de regresión.
246     # @param self El puntero al objeto.
247     # @param f Función que se quiere integrar.
248     # @param extremos Hipercubo sobre el cual se quiere integrar la función.
249     # @param nmuestras Número de muestras a utilizar para integrar.
250     # @param n_splits Número de pliegues.
251     # @param minLeaf=1 El mínimo número de muestras requeridas para se un nodo hoja
252     .
253     # @param depth=None Profundidad máxima del árbol.

```

```

248 # @return 3-upla formada por la integral, una estimación del error absoluto con
    una confianza del 99% y una métrica que indica si la aproximación es buena.
250 def StackMonteCarloArbol(self, f, extremos, nmuestras, n_splits, minLeaf=1, depth=None
    ):
    muestras = [[np.random.uniform(extremoA, extremoB) for [extremoA, extremoB] in
    extremos] for i in range(nmuestras)]
    muestrasT = np.array([np.array(x) for x in muestras])
252 fmuestras = np.array([f(x) for x in muestras])

254 mf = np.mean(fmuestras)
    #varf = np.var(fmuestras)

256 kf = KFold(n_splits=n_splits)
    #kf.get_n_splits(muestrasT)

258 #print(kf)
    modelos = []
    scores = []
    integrales = []
264 sumasgis = []
    KFold(n_splits=5, random_state=None, shuffle=False)
266 for train_index, test_index in kf.split(muestrasT):

268     X_train, X_test = muestrasT[train_index], muestrasT[test_index]
    y_train, y_test = fmuestras[train_index], fmuestras[test_index]
270 modelo = Arbol(depth=depth, extrems=extremos, minLeaf=minLeaf)
    modelo.train(X_train, y_train)
272 modelos.append(modelo)

274 scores.append(modelo.score(X_test, y_test))
    integrales.append(modelo.integral)

276 sumasgis.append(np.sum(modelo.predict(X_test.astype('float32'))))

278 mediag = np.sum(sumasgis) / nmuestras
    sumaVarg = []
    sumaCovar = []

282 i=0

284 for train_index, test_index in kf.split(muestrasT):
286     #print("TRAIN:", train_index, "TEST:", test_index)
    X_train, X_test = muestrasT[train_index], muestrasT[test_index]
288 y_train, y_test = fmuestras[train_index], fmuestras[test_index]
    auxg = modelos[i].predict(X_test.astype('float32')) - mediag
290 auxf = np.array([f(x)-mf for x in muestrasT[test_index]])
    #para la varianza de g
292 sumaVarg.append(np.sum(np.power(auxg, 2)))
    #para la covarianza
294 sumaCovar.append(np.sum(np.multiply(auxg, auxf)))
    i = i + 1

296 varg = np.sum(sumaVarg)/nmuestras
298 covar = np.sum(sumaCovar)/nmuestras

300 alfa = covar/varg

302 i = 0
304 estimaciones = []

```



```

std_estimaciones = []
306 for train_index, test_index in kf.split(muestrasT):
    X_train, X_test = muestrasT[train_index], muestrasT[test_index]
308 y_train, y_test = fmuestras[train_index], fmuestras[test_index]

    pf = y_test
    pg = modelos[i].predict(X_test.astype('float32'))

312
    resta = pf - (alfa * pg)
    mr = np.mean(resta)
314 sr = np.std(resta)
    estimaciones.append(alfa * modelos[i].integral + self.volumen(extremos)
316 * mr)
    std_estimaciones.append(self.volumen(extremos)*sr)

318
NMT = nmuestras / n_splits
320 std_estimaciones = np.array(std_estimaciones)
    errores_estimaciones_previstos = 2.58*std_estimaciones/(math.sqrt(NMT))
322
    fmc = np.mean(fmuestras * self.volumen(extremos))
324 smc = np.std(fmuestras * self.volumen(extremos))
    sdeur = 2.58* smc / math.sqrt(nmuestras)
326

    L = np.abs( np.array([ m.integral for m in modelos]) - fmc )/sdeur
328
330 return (np.mean(estimaciones), np.max(errores_estimaciones_previstos), np.max(
L))

```

Clase que encapsula varios métodos de integración vistos en el trabajo

Bibliografía

- [1] Félix Belzunce, José María Ruiz. Guiones de clase de la asignatura Inferencia Estadística. Facultad de Matemáticas. Universidad de Murcia.
- [2] Joshua Bendavid. Efficient Monte Carlo Integration Using Boosted Decision Trees and Generative Deep Neural Networks. Lauritsen Laboratory for High Energy Physics, California Institute of Technology. June 2017. <https://arxiv.org/abs/1707.00028>
- [3] Michael Evans, Tim Swartz. *Approximating Integrals via Monte Carlo and Deterministic Methods*. Oxford University Press, first edition, 2000.
- [4] José Ignacio Illana. Métodos Monte Carlo. Departamento de Física Teórica y del Cosmos. Universidad de Granada. <https://www.ugr.es/~jillana/Docencia/FM/mc.pdf>
- [5] G. P. Lepage. A New Algorithm for Adaptive Multidimensional Integration. *Journal of Computational Physics* 27 (1978) 192 – 203. <http://www.slac.stanford.edu/pubs/slacpubs/1750/slac-pub-1839.pdf>
- [6] G. P. Lepage. VEGAS - an adaptive multi-dimensional integration program. Newman Laboratory of Nuclear Studies. Cornell University. March 1980. <http://cds.cern.ch/record/123074/files/clns-447.pdf>
- [7] G. P. Lepage. vegas Documentation. <https://www.acsu.buffalo.edu/~phygons/cp2/topic6/vegas.pdf>
- [8] David Neal. Determining Sample Sizes for Monte Carlo Integration. *The College Mathematics Journal*, Vol. 24, No. 3 (May, 1993), pp. 254-259
- [9] A.T. Patera and M. Yano. Monte Carlo Integration... in a Nutshell. https://ocw.mit.edu/courses/mechanical-engineering/2-086-numerical-computation-for-mechanical-engineers-fall-2014/nutshells-guis/MIT2_086F14_Monte_Carlo.pdf
- [10] W. H. Press, S. A. Teukolsky, W. T. Vetterling, and B. P. Flannery. *Numerical Recipes in C*. Cambridge University Press, second edition, 1992.
- [11] W. H. Press and Glennys R. Farrar. Recursive Stratified Sampling For Multidimensional Monte Carlo Integration. *Computer in Physics*. Vol 4. Number 2. pp. 190-195, 1990. <https://aip.scitation.org/doi/pdf/10.1063/1.4822899?class=pdf>
- [12] Brendan Tracey, David Wolpert, Juan J. Alonso. Using Supervised Learning to Improve Monte Carlo Integral Estimation. 13th AIAA Non-Deterministic Approaches Conference, Denver, CO, April 2011, AIAA Paper 2011-1843. <https://arxiv.org/abs/1108.4879>

-
- [13] Procopio Zoroa Terol, Noemí Zoroa Alonso. *Elementos de probabilidades*. Diego Marín, primera edición, 2008.
 - [14] PyPI. vegas 3.4 <https://pypi.org/project/vegas/>
 - [15] Karoliina Oksanen. Python implementation of the MISER Monte Carlo method. <https://github.com/karoliina/MISER>
 - [16] scikit-learn. Machine Learning in Python. <https://scikit-learn.org/stable/>
 - [17] Scikit Learn, Decision Trees. <https://scikit-learn.org/stable/modules/tree.html#tree>
 - [18] Scikit Learn, sklearn.tree.DecisionTreeRegressor. <https://scikit-learn.org/stable/modules/generated/sklearn.tree.DecisionTreeRegressor.html#sklearn.tree.DecisionTreeRegressor>
 - [19] Numpy. <https://www.numpy.org/>
 - [20] Matplotlib. <https://matplotlib.org/>
 - [21] math — Mathematical functions. <https://docs.python.org/3/library/math.html>